PRACTICAL NO. 1

Aim: Implement a deep neural network from scratch using TensorFlow or PyTorch, gaining hands-on experience in building complex neural architectures.

Theory:

Introduction to Deep Learning: - Deep learning is a specialized branch of machine learning that enables computers to learn complex patterns from large amounts of data using artificial neural networks with multiple hidden layers. Unlike traditional machine learning, which often requires manual feature extraction, deep learning automatically identifies relevant features during training. This capability makes it highly effective for tasks such as image classification, speech recognition, natural language processing, recommendation systems, and autonomous driving. With the availability of large datasets, powerful GPUs, and advanced algorithms, deep learning has become one of the most important technologies in modern artificial intelligence and data science.

Deep Neural Networks (DNNs): - A Deep Neural Network (DNN) is the primary architecture used in deep learning. It consists of an input layer, multiple hidden layers, and an output layer, where neurons process information through weights, biases, and activation functions. During training, the network learns by minimizing errors using backpropagation and optimization algorithms such as Gradient Descent or Adam. The multiple hidden layers enable the network to extract increasingly complex features from the input data, resulting in highly accurate predictions. DNNs are widely used in computer vision, healthcare, finance, robotics, and many other real-world applications.

TensorFlow and PyTorch: - TensorFlow and PyTorch are the two most popular open-source frameworks used for implementing deep learning models. TensorFlow, developed by Google, provides a powerful ecosystem along with the Keras API, allowing developers to build neural networks quickly using predefined layers and functions. PyTorch, developed by Meta AI, is known for its flexibility, dynamic computation graph, and Python-friendly syntax, making it a preferred choice for research and experimentation. Both frameworks support GPU acceleration, automatic differentiation, and a wide range of neural network architectures, making them suitable for building and training complex deep learning models efficiently.

Importing TensorFlow and PyTorch (Model Syntax): - Before implementing a deep neural network, the required libraries must be imported into the Python program. In TensorFlow, models are commonly created using the `Sequential` class or the Functional API by importing modules such as `tensorflow.keras.models` and `tensorflow.keras.layers`. In PyTorch, models are defined by creating a class that inherits from `torch.nn.Module`, while layers and optimization algorithms are imported from `torch.nn` and `torch.optim`. The basic import statements are:

```
# TensorFlow
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
```

```
# PyTorch
import torch
import torch.nn as nn
import torch.optim as optim
```

Algorithm:

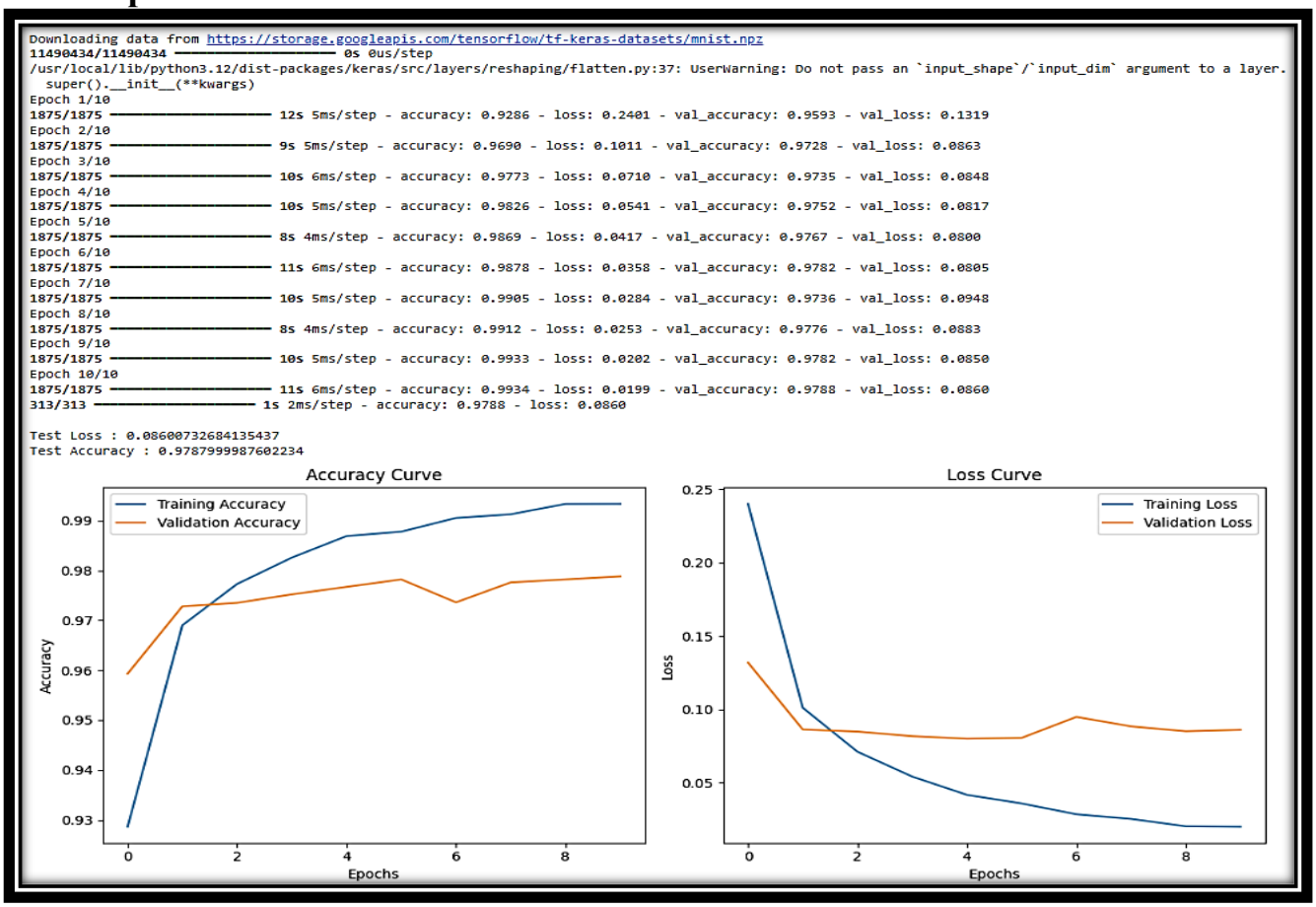
1. Import the required libraries: TensorFlow, Keras modules, and Matplotlib.
2. Load the built-in MNIST handwritten digits dataset.
3. Normalize the pixel values by dividing each pixel by 255.0.
4. Create a Sequential Deep Neural Network consisting of:
    a. Flatten input layer
    b. Dense hidden layer with 128 neurons and ReLU activation
    c. Dense hidden layer with 64 neurons and ReLU activation
    d. Output layer with 10 neurons and Softmax activation.
5. Compile the model using:
    a. Adam optimizer
    b. Sparse Categorical Crossentropy loss function
    c. Accuracy as the evaluation metric.
6. Train the model for 10 epochs using the training dataset and validate it using the test dataset.
7. Evaluate the trained model on the test dataset and display the test accuracy and loss.
8. Plot the Training vs Validation Accuracy curve.
9. Plot the Training vs Validation Loss curve.
10. Display the graphs and analyze the model's performance.

Source Code:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Flatten
import matplotlib.pyplot as plt
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.mnist.load_data()
x_train = x_train / 255.0
x_test = x_test / 255.0
model = Sequential([
    Flatten(input_shape=(28, 28)),
    Dense(128, activation='relu'),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')])
model.compile(
    optimizer='adam',
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
history = model.fit(
    x_train,
    y_train,
    epochs=10,
    validation_data=(x_test, y_test))
```

```
loss, accuracy = model.evaluate(x_test, y_test)
print("\nTest Loss :", loss)
print("Test Accuracy :", accuracy)
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Training
Accuracy')
plt.plot(history.history['val_accuracy'], label='Validation
Accuracy')
plt.title("Accuracy Curve")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()
plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title("Loss Curve")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.tight_layout()
plt.show()
```

Output:



Learning Outcome:

PRACTICAL NO. 2

Aim: Utilize pre-trained models and perform transfer learning to solve real-world problems efficiently.

Theory:

Transfer Learning: Transfer learning is a machine learning technique in which knowledge acquired by a model while solving one problem is reused to solve another related problem. In conventional deep learning, a neural network is trained from scratch using a large amount of labeled data, which can require significant computational resources and training time. Transfer learning reduces these requirements by starting with a model that has already learned useful features from a large dataset. In image classification, the initial layers of a convolutional neural network generally learn generic features such as edges, curves, textures, and colors, while deeper layers learn more task-specific patterns. These learned representations can therefore be transferred to a new dataset and adapted for a different classification task.

Pre-trained Models: A pre-trained model is a deep neural network that has previously been trained on a large dataset and whose learned weights are reused for a new application. Several important pre-trained convolutional neural networks are widely used for transfer learning. VGG16 uses a simple architecture containing sequential convolutional layers and is known for its strong feature extraction capability. ResNet50 introduces residual connections, allowing very deep networks to be trained effectively. InceptionV3 uses multiple convolutional operations with different filter sizes to capture features at different scales. MobileNetV2 is designed for computational efficiency and uses depthwise separable convolutions and inverted residual blocks. EfficientNetB0 provides a strong balance between accuracy and computational efficiency through systematic scaling of network depth, width, and image resolution.

Fashion MNIST Dataset: Fashion MNIST is a benchmark image classification dataset provided as a built-in dataset in TensorFlow. It contains 70,000 grayscale images, divided into 60,000 training images and 10,000 testing images. Each image has a resolution of 28 × 28 pixels and belongs to one of ten fashion categories: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. Although the images are relatively small and simple, Fashion MNIST represents a practical multi-class image classification problem. Since the pre-trained model used in this experiment expects three-channel images of a larger spatial resolution, the grayscale images are converted into RGB format and resized before being passed to the network.

For this experiment, MobileNetV2 is selected as the pre-trained model because it provides an effective compromise between feature extraction capability, computational cost, and model size. The network is initially trained on the ImageNet dataset, which contains millions of natural images belonging to numerous categories. Its convolutional base has already learned general visual representations that can be useful for other image-related tasks. During transfer learning, the original ImageNet classification layer is removed and the convolutional base is kept frozen. A new classification head consisting of Global Average Pooling, Dropout, and a Dense layer with ten softmax outputs is added. Consequently, only the newly introduced layers need to learn the Fashion MNIST-specific classification task.

Transfer Learning Procedure and Advantages: The transfer learning procedure begins by loading MobileNetV2 with its pre-trained ImageNet weights while excluding its original classification layer. Fashion MNIST images are normalized, converted from grayscale to three-channel RGB images, and resized to 96 × 96 pixels. The processed images are then supplied to the frozen MobileNetV2 feature extractor. The extracted representations are passed through Global Average Pooling and Dropout before reaching the final Dense classification layer. The model is trained using the Adam optimizer and sparse categorical cross-entropy loss. This methodology demonstrates the practical advantages of transfer learning, including reduced training time, lower computational requirements, improved feature extraction, and the ability to achieve useful performance even when the target dataset is considerably smaller than the dataset used for initial pre-training.

Model	Main Feature	Major Advantage
VGG16	Sequential convolutional architecture	Simple and effective feature extraction
ResNet50	Residual/skip connections	Enables training of very deep networks
InceptionV3	Multi-scale convolutional operations	Captures features at different spatial scales
MobileNetV2	Depthwise separable convolutions	Lightweight and computationally efficient
EfficientNetB0	Compound model scaling	Good accuracy-to-computation ratio

Algorithm:

1. Import TensorFlow and the required Keras libraries and layers.
2. Load the Fashion MNIST dataset using TensorFlow's built-in fashion_mnist.load_data() function.
3. Divide the dataset into training images, training labels, testing images, and testing labels.
4. Convert the image pixel values from integer format to floating-point values.
5. Normalize all pixel values from the range 0–255 to 0–1.
6. Add a channel dimension to the grayscale images.
7. Convert each grayscale image into a three-channel RGB image because MobileNetV2 requires three input channels.
8. Resize all images from 28 × 28 to 96 × 96 pixels.
9. Load MobileNetV2 with ImageNet pre-trained weights and remove its original classification layer using include_top=False.
10. Freeze the MobileNetV2 convolutional base so that its pre-trained weights are not modified during training.
11. Add a Global Average Pooling 2D layer to convert the extracted feature maps into a compact feature representation.
12. Add a Dropout layer to reduce the possibility of overfitting.

13. Add a Dense layer with 10 neurons and softmax activation corresponding to the ten Fashion MNIST classes.
14. Compile the model using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.
15. Train the model using the Fashion MNIST training data for the specified number of epochs.
16. Use a validation split to monitor model performance during training.
17. Evaluate the trained model using the unseen Fashion MNIST test dataset.
18. Display the final test loss and classification accuracy.

Source Code:

```
import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.applications import MobileNetV2
print("TensorFlow Version:", tf.__version__)
(x_train, y_train), (x_test, y_test) =
tf.keras.datasets.fashion_mnist.load_data()
print("\nOriginal Dataset")
print("Training images :", x_train.shape)
print("Training labels :", y_train.shape)
print("Testing images  :", x_test.shape)
print("Testing labels  :", y_test.shape)
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = tf.expand_dims(x_train, axis=-1)
x_test = tf.expand_dims(x_test, axis=-1)
x_train = tf.image.grayscale_to_rgb(x_train)
x_test = tf.image.grayscale_to_rgb(x_test)
x_train = tf.image.resize(x_train, (96, 96))
x_test = tf.image.resize(x_test, (96, 96))
print("\nAfter Preprocessing")
print("Training images :", x_train.shape)
print("Testing images  :", x_test.shape)
base_model = MobileNetV2(
    weights="imagenet",
    include_top=False,
    input_shape=(96, 96, 3))
base_model.trainable = False
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dropout(0.3),
    layers.Dense(10, activation="softmax")])
model.compile(
    optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])
```

```
print("\nModel Architecture")
model.summary()
print("\nStarting Training...")
history = model.fit(
    x_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1,
    verbose=1)
test_loss, test_accuracy = model.evaluate(
    x_test,
    y_test,
    verbose=1)
print("Test Loss     :", round(test_loss, 4))
print("Test Accuracy :", round(test_accuracy * 100, 2), "%")
class_names = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"]
print("\nFashion MNIST Classes:")
for i, name in enumerate(class_names):
    print(i, "->", name)
```

Output:

```
TensorFlow Version: 2.20.0
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step

Original Dataset
Training images : (60000, 28, 28)
Training labels : (60000,)
Testing images  : (10000, 28, 28)
Testing labels  : (10000,)

After Preprocessing
Training images : (60000, 96, 96, 3)
Testing images  : (10000, 96, 96, 3)
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet_v2/mobilenet_v2_weights_tf_dim_ordering_tf_kernels_1.0_96_no_top.h5
9406464/9406464 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
```

Model Architecture

Model: "sequential"

Layer (type)	Output Shape	Param #
mobilenetv2_1.00_96 (Functional)	(None, 3, 3, 1280)	2,257,984
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	0
dropout (Dropout)	(None, 1280)	0
dense (Dense)	(None, 10)	12,810

Total params: 2,270,794 (8.66 MB)
Trainable params: 12,810 (50.04 KB)
Non-trainable params: 2,257,984 (8.61 MB)

```
Starting Training...
Epoch 1/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 51s 75ms/step - accuracy: 0.8048 - loss: 0.5551 - val_accuracy: 0.8795 - val_loss: 0.3313
Epoch 2/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.8680 - loss: 0.3670 - val_accuracy: 0.8880 - val_loss: 0.3130
Epoch 3/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.8739 - loss: 0.3434 - val_accuracy: 0.8903 - val_loss: 0.2988
Epoch 4/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.8798 - loss: 0.3308 - val_accuracy: 0.8932 - val_loss: 0.2971
Epoch 5/5
422/422 ━━━━━━━━━━━━━━━━━━━━ 12s 28ms/step - accuracy: 0.8834 - loss: 0.3216 - val_accuracy: 0.8933 - val_loss: 0.2959
313/313 ━━━━━━━━━━━━━━━━━━━━ 24s 42ms/step - accuracy: 0.8832 - loss: 0.3191
Test Loss     : 0.3191
Test Accuracy : 88.32 %

Fashion MNIST Classes:
0 -> T-shirt/top
1 -> Trouser
2 -> Pullover
3 -> Dress
4 -> Coat
5 -> Sandal
6 -> Shirt
7 -> Sneaker
8 -> Bag
9 -> Ankle boot
```

Learning Outcome:

PRACTICAL NO. 3

Aim: Implement a GAN to generate synthetic data and explore its applications in image generation and data augmentation.

Theory:

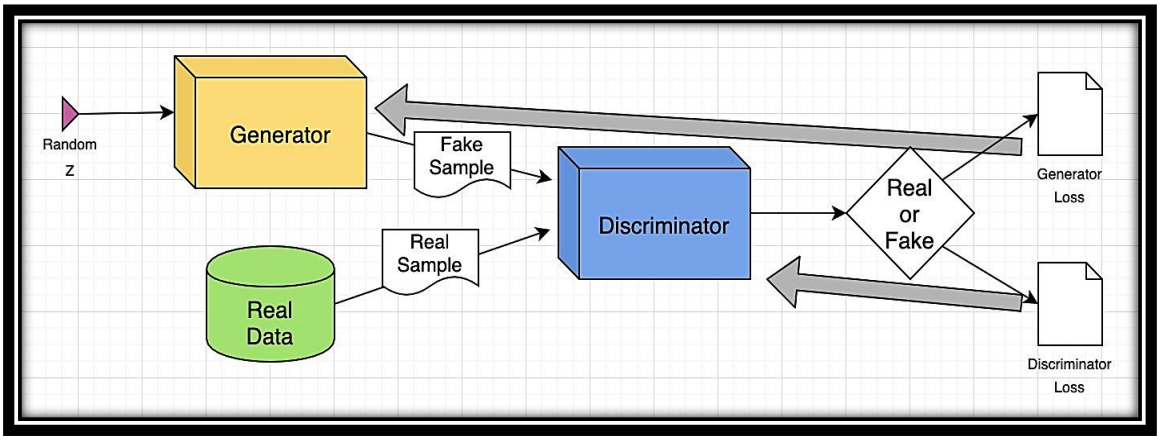
Advanced Machine Learning: Advanced Machine Learning (AML) is a specialized area of machine learning that focuses on developing models capable of learning complex patterns and representations from data. It includes techniques such as deep learning, generative models, transfer learning, reinforcement learning, and other advanced neural network architectures. Unlike traditional machine learning methods, advanced learning techniques can automatically discover useful features from raw and unstructured data. Deep learning models are particularly effective for image, speech, and text-based applications because they can learn hierarchical representations. Advanced Machine Learning is widely applied in computer vision, natural language processing, recommendation systems, healthcare, finance, robotics, and autonomous systems. In this experiment, an advanced generative learning technique called Generative Adversarial Network is implemented for synthetic image generation.

Generative Adversarial Network (GAN): A Generative Adversarial Network (GAN) is a deep learning architecture used to generate new data samples that resemble samples from a given training dataset. A GAN consists of two neural networks called the Generator and the Discriminator, which are trained simultaneously in an adversarial process. The Generator receives a random noise vector as input and attempts to produce realistic synthetic samples. The Discriminator receives both real and generated samples and attempts to distinguish between them. During training, the Generator learns to create increasingly realistic samples, while the Discriminator learns to identify generated samples more accurately. The competition between the two networks enables the Generator to gradually learn the underlying data distribution and produce synthetic samples similar to the original training data.

Deep Convolutional GAN (DCGAN): A Deep Convolutional Generative Adversarial Network (DCGAN) is a GAN architecture that uses convolutional neural network operations to improve image generation. In this experiment, a lightweight DCGAN-style architecture is implemented using TensorFlow and Keras. The Generator begins with a 100-dimensional random noise vector and uses a Dense layer to create a feature representation, which is reshaped into a 7×7 feature map. Transposed convolutional layers then progressively increase the spatial dimensions until a 28×28 grayscale image is produced. The Discriminator uses convolutional layers with downsampling to extract spatial features from input images. LeakyReLU and Dropout layers are used to improve learning and reduce overfitting. This architecture is more suitable for image generation than a purely fully connected GAN.

MNIST Dataset and Synthetic Image Generation: The MNIST dataset is a standard dataset containing grayscale images of handwritten digits from 0 to 9. Each image has a resolution of 28×28 pixels, making MNIST suitable for lightweight experiments involving image generation. In this experiment, the complete training set containing 60,000 images is used to train the GAN. The pixel values are converted from the original range of 0–255 to the range -1 to 1 because the Generator uses the Tanh activation function at its output. The

Generator learns to transform random 100-dimensional noise vectors into synthetic 28 × 28 handwritten digit images. After training, 20 fixed random noise vectors are provided to the Generator, producing 20 synthetic images that are displayed in a 4 × 5 grid.



Applications of GANs and Data Augmentation: GANs have several important applications in artificial intelligence and machine learning. They can generate synthetic images, perform image-to-image translation, enhance image resolution, restore damaged images, and create additional training data. One important application is data augmentation, where generated samples can supplement real training data when the available dataset is small or lacks sufficient diversity. GANs are also useful for generating simulated medical images, creative content, and other synthetic datasets. In this experiment, the Generator creates new handwritten digit images based on patterns learned from MNIST. These synthetic images demonstrate how generative models can be used to produce additional data samples, which can potentially be combined with real images to improve the robustness and diversity of machine learning training datasets.

Algorithm:

1. Import TensorFlow, Keras, NumPy, and Matplotlib, then load the MNIST dataset and extract the 60,000 training images while ignoring the labels.
2. Convert images to floating-point format, normalize pixels from 0–255 to −1 to 1, add the channel dimension, and create a shuffled TensorFlow dataset with batches of 128.
3. Define the Generator with a 100-dimensional noise input, a Dense layer of 7 × 7 × 128 neurons, Batch Normalization, LeakyReLU, and reshape it into a 7 × 7 × 128 feature map.
4. Use Transposed Convolution layers in the Generator to increase the image size from 7 × 7 to 14 × 14 and finally 28 × 28, producing a grayscale image using Tanh activation.
5. Define the Discriminator with convolutional layers using stride 2, LeakyReLU, and Dropout, followed by Flatten and a Sigmoid output to classify images as real / generated.
6. Use Binary Cross-Entropy loss and separate Adam optimizers with a learning rate of 0.0002 for both networks.
7. During each batch, generate fake images from random noise, train the Discriminator using real and fake images, and train the Generator to make its generated images appear real.
8. Repeat the adversarial training process for 30 epochs using the complete MNIST dataset.
9. Generate 20 synthetic images using fixed random noise vectors and convert their pixel values from −1 to 1 back to 0–1.
10. Display the 20 generated images in a 4 × 5 grid using Matplotlib and print their shape and total number.

Source Code:

```
import tensorflow as tf
from tensorflow.keras import layers
import numpy as np
import matplotlib.pyplot as plt
print("TensorFlow Version:", tf.__version__)
(x_train, _), (_, _) = tf.keras.datasets.mnist.load_data()
print("\nMNIST Dataset")
print("Training images :", x_train.shape)
x_train = x_train.astype("float32")
x_train = (x_train - 127.5) / 127.5
x_train = np.expand_dims(x_train, axis=-1)
BATCH_SIZE = 128
dataset = tf.data.Dataset.from_tensor_slices(x_train)
dataset =
dataset.shuffle(60000).batch(BATCH_SIZE).prefetch(tf.data.AUTOTUNE)
generator = tf.keras.Sequential([
    layers.Input(shape=(100,)),
    layers.Dense(7 * 7 * 128, use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Reshape((7, 7, 128)),
    layers.Conv2DTranspose(
        64, (5, 5),
        strides=(1, 1),
        padding="same",
        use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(
        32, (5, 5),
        strides=(2, 2),
        padding="same",
        use_bias=False),
    layers.BatchNormalization(),
    layers.LeakyReLU(),
    layers.Conv2DTranspose(
        1, (5, 5),
        strides=(2, 2),
        padding="same",
        activation="tanh")], name="Generator")
discriminator = tf.keras.Sequential([
    layers.Input(shape=(28, 28, 1)),
    layers.Conv2D(
        64, (5, 5),
        strides=(2, 2),
        padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
```

```
    layers.Dropout(0.3),
    layers.Conv2D(
        128, (5, 5),
        strides=(2, 2),
        padding="same"),
    layers.LeakyReLU(negative_slope=0.2),
    layers.Dropout(0.3),
    layers.Flatten(),
    layers.Dense(1, activation="sigmoid")], name="Discriminator")
print("\nGenerator Architecture")
generator.summary()
print("\nDiscriminator Architecture")
discriminator.summary()
loss_function = tf.keras.losses.BinaryCrossentropy()
generator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002,
    beta_1=0.5)
discriminator_optimizer = tf.keras.optimizers.Adam(
    learning_rate=0.0002,
    beta_1=0.5)
seed = tf.random.normal([20, 100])
@tf.function
def train_step(real_images):
    noise = tf.random.normal(
        [tf.shape(real_images)[0], 100])
    fake_images = generator(
        noise,
        training=True)
    with tf.GradientTape() as disc_tape:
        real_output = discriminator(
            real_images,
            training=True)
        fake_output = discriminator(
            fake_images,
            training=True)
        real_loss = loss_function(
            tf.ones_like(real_output),
            real_output)
        fake_loss = loss_function(
            tf.zeros_like(fake_output),
            fake_output)
        discriminator_loss = (
            real_loss + fake_loss)
    disc_gradients = disc_tape.gradient(
        discriminator_loss,
        discriminator.trainable_variables)
    discriminator_optimizer.apply_gradients(
        zip(disc_gradients,
```

```
            discriminator.trainable_variables))
    with tf.GradientTape() as gen_tape:
        noise = tf.random.normal(
            [tf.shape(real_images)[0], 100])
        generated_images = generator(
            noise,
            training=True)
        generated_output = discriminator(
            generated_images,
            training=True)
        generator_loss = loss_function(
            tf.ones_like(generated_output),
            generated_output)
    gen_gradients = gen_tape.gradient(
        generator_loss,
        generator.trainable_variables)
    generator_optimizer.apply_gradients(
        zip(gen_gradients,
            generator.trainable_variables))
    return generator_loss, discriminator_loss
    EPOCHS = 30
print("\nStarting GAN Training...")
for epoch in range(EPOCHS):
    gen_loss_avg = tf.keras.metrics.Mean()
    disc_loss_avg = tf.keras.metrics.Mean()
    for batch in dataset:
        gen_loss, disc_loss = train_step(batch)
        gen_loss_avg.update_state(gen_loss)
        disc_loss_avg.update_state(disc_loss)
    print(
        f"Epoch {epoch + 1}/{EPOCHS} - "
        f"Generator Loss: {gen_loss_avg.result():.4f}, "
        f"Discriminator Loss: {disc_loss_avg.result():.4f}")
print("\nTraining Completed!")
generated_images = generator(
    seed,
    training=False)
generated_images = (
    generated_images + 1) / 2
plt.figure(figsize=(10, 8))
for i in range(20):
    plt.subplot(4, 5, i + 1)
    plt.imshow(
        generated_images[i, :, :, 0],
        cmap="gray")
    plt.axis("off")
    plt.title(f"Image {i + 1}")
plt.suptitle(
```

```
    "20 Synthetic MNIST Images Generated by GAN",
    fontsize=14)
plt.tight_layout()
plt.show()
print("\nGenerated Image Shape:", generated_images.shape)
print("Total Synthetic Images Generated:", len(generated_images))
```

Output:

```
TensorFlow Version: 2.20.0

MNIST Dataset
Training images : (60000, 28, 28)

Generator Architecture
Model: "Generator"
```

Layer (type)	Output Shape	Param #
dense_12 (Dense)	(None, 6272)	627,200
batch_normalization (BatchNormalization)	(None, 6272)	25,088
leaky_re_lu_4 (LeakyReLU)	(None, 6272)	0
reshape_2 (Reshape)	(None, 7, 7, 128)	0
conv2d_transpose (Conv2DTranspose)	(None, 7, 7, 64)	204,800
batch_normalization_1 (BatchNormalization)	(None, 7, 7, 64)	256
leaky_re_lu_5 (LeakyReLU)	(None, 7, 7, 64)	0
conv2d_transpose_1 (Conv2DTranspose)	(None, 14, 14, 32)	51,200
batch_normalization_2 (BatchNormalization)	(None, 14, 14, 32)	128
leaky_re_lu_6 (LeakyReLU)	(None, 14, 14, 32)	0
conv2d_transpose_2 (Conv2DTranspose)	(None, 28, 28, 1)	801

```
Total params: 909,473 (3.47 MB)
Trainable params: 896,737 (3.42 MB)
Non-trainable params: 12,736 (49.75 KB)
```

Discriminator Architecture

Model: "Discriminator"

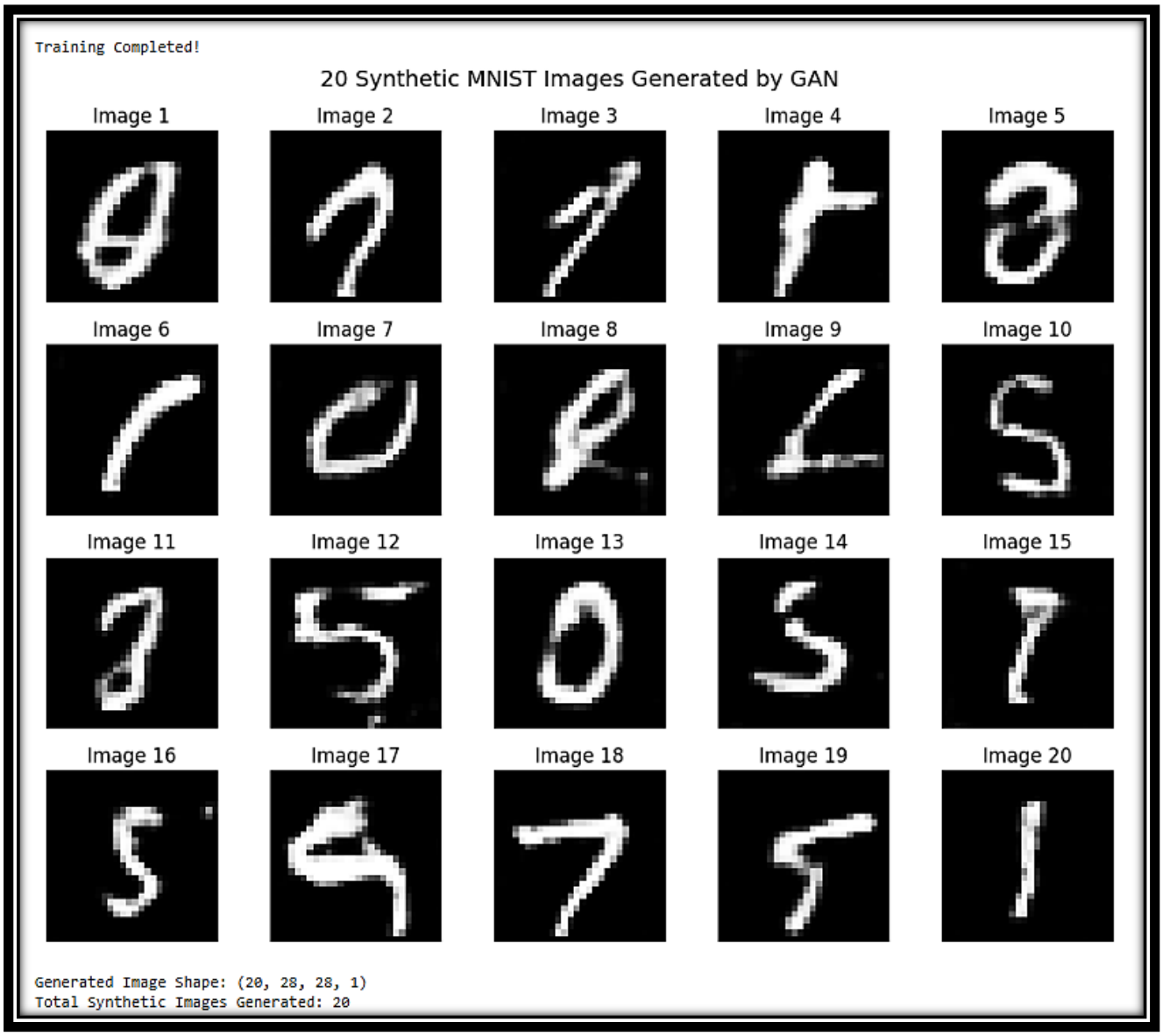
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 14, 14, 64)	1,664
leaky_re_lu_7 (LeakyReLU)	(None, 14, 14, 64)	0
dropout (Dropout)	(None, 14, 14, 64)	0
conv2d_1 (Conv2D)	(None, 7, 7, 128)	204,928
leaky_re_lu_8 (LeakyReLU)	(None, 7, 7, 128)	0
dropout_1 (Dropout)	(None, 7, 7, 128)	0
flatten_2 (Flatten)	(None, 6272)	0
dense_13 (Dense)	(None, 1)	6,273

Total params: 212,865 (831.50 KB)

Trainable params: 212,865 (831.50 KB)

Non-trainable params: 0 (0.00 B)

```
Starting GAN Training...
Epoch 1/30 - Generator Loss: 0.8957, Discriminator Loss: 1.1492
Epoch 2/30 - Generator Loss: 0.8510, Discriminator Loss: 1.2720
Epoch 3/30 - Generator Loss: 0.8984, Discriminator Loss: 1.2216
Epoch 4/30 - Generator Loss: 0.8387, Discriminator Loss: 1.2806
Epoch 5/30 - Generator Loss: 0.7999, Discriminator Loss: 1.3042
Epoch 6/30 - Generator Loss: 0.7984, Discriminator Loss: 1.3113
Epoch 7/30 - Generator Loss: 0.8211, Discriminator Loss: 1.3089
Epoch 8/30 - Generator Loss: 0.8190, Discriminator Loss: 1.2970
Epoch 9/30 - Generator Loss: 0.8222, Discriminator Loss: 1.2883
Epoch 10/30 - Generator Loss: 0.8505, Discriminator Loss: 1.2829
Epoch 11/30 - Generator Loss: 0.8505, Discriminator Loss: 1.2721
Epoch 12/30 - Generator Loss: 0.8623, Discriminator Loss: 1.2698
Epoch 13/30 - Generator Loss: 0.8648, Discriminator Loss: 1.2607
Epoch 14/30 - Generator Loss: 0.8791, Discriminator Loss: 1.2605
Epoch 15/30 - Generator Loss: 0.8793, Discriminator Loss: 1.2511
Epoch 16/30 - Generator Loss: 0.8975, Discriminator Loss: 1.2523
Epoch 17/30 - Generator Loss: 0.8878, Discriminator Loss: 1.2465
Epoch 18/30 - Generator Loss: 0.9137, Discriminator Loss: 1.2418
Epoch 19/30 - Generator Loss: 0.8995, Discriminator Loss: 1.2372
Epoch 20/30 - Generator Loss: 0.9088, Discriminator Loss: 1.2392
Epoch 21/30 - Generator Loss: 0.9138, Discriminator Loss: 1.2358
Epoch 22/30 - Generator Loss: 0.9183, Discriminator Loss: 1.2362
Epoch 23/30 - Generator Loss: 0.9240, Discriminator Loss: 1.2345
Epoch 24/30 - Generator Loss: 0.9190, Discriminator Loss: 1.2303
Epoch 25/30 - Generator Loss: 0.9276, Discriminator Loss: 1.2301
Epoch 26/30 - Generator Loss: 0.9304, Discriminator Loss: 1.2296
Epoch 27/30 - Generator Loss: 0.9358, Discriminator Loss: 1.2357
Epoch 28/30 - Generator Loss: 0.9262, Discriminator Loss: 1.2270
Epoch 29/30 - Generator Loss: 0.9346, Discriminator Loss: 1.2330
Epoch 30/30 - Generator Loss: 0.9317, Discriminator Loss: 1.2307
```



Learning Outcome:

PRACTICAL NO. 4

Aim: Apply NLP techniques to process and analyze textual data, including sentiment analysis and named entity recognition.

Theory:

Advanced Machine Learning: Advanced Machine Learning (AML) is an area of machine learning that focuses on developing models capable of learning complex patterns and relationships from large and diverse datasets. It includes techniques such as deep learning, natural language processing, generative models, computer vision, reinforcement learning, and transfer learning. AML techniques are particularly useful for handling unstructured information such as images, audio, and text. Natural Language Processing is an important application of AML that enables computers to understand and analyze human language. In this experiment, AML techniques are applied to Twitter data to classify tweets according to their sentiment and extract meaningful information from the text using Named Entity Recognition.

Natural Language Processing: Natural Language Processing (NLP) is a branch of Artificial Intelligence and Machine Learning that enables computers to process, understand, and analyze human language. Textual information collected from social media, websites, documents, and communication platforms is generally unstructured and cannot be directly processed by most machine learning algorithms. NLP converts this text into a suitable representation for computational analysis. Common NLP tasks include text classification, sentiment analysis, machine translation, text summarization, information extraction, and Named Entity Recognition. In this experiment, NLP is applied to tweets from the Sentiment140 dataset. The tweets are cleaned and transformed into numerical features using TF-IDF before being classified into positive or negative sentiment.

NLP Processing Steps: The NLP process generally begins with collecting and loading textual data into a suitable data structure. The raw text is then preprocessed to reduce unnecessary information and make it more suitable for analysis. Basic preprocessing may include converting text to lowercase, removing URLs, user mentions, punctuation, and special characters, followed by tokenization and stop-word removal. After preprocessing, the cleaned text is converted into numerical features using techniques such as Bag-of-Words or TF-IDF. A machine learning algorithm can then learn patterns from these features and perform the required task. In this experiment, tweets are cleaned using basic NLP techniques, converted into TF-IDF vectors, classified using Logistic Regression, and finally analyzed using a pretrained NLP model for Named Entity Recognition.

Sentiment Analysis and Named Entity Recognition: Sentiment Analysis is an NLP technique used to determine the emotional orientation expressed in a piece of text. It is commonly used to classify text as positive, negative, or neutral. In this experiment, Logistic Regression is trained on TF-IDF representations of tweets to classify their sentiment as positive or negative. Named Entity Recognition (NER) is another NLP technique that identifies specific entities in textual data and assigns categories to them. These categories may include people, organizations, locations, dates, and other named objects. A pretrained spaCy English model is used for NER in this experiment. Therefore, sentiment analysis

determines the overall opinion expressed in a tweet, while NER extracts important named entities from the same textual data.

Applications of NLP: NLP has numerous real-world applications because large amounts of information are generated in textual form every day. Sentiment analysis can be used for monitoring customer opinions, analyzing product reviews, measuring public feedback, tracking brand perception, and studying social media trends. Named Entity Recognition can automatically extract names of people, organizations, locations, products, and other entities from large collections of text. NLP is also used in chatbots, virtual assistants, search engines, spam detection, recommendation systems, document classification, and information retrieval. In this experiment, Twitter data provides a practical example of NLP by allowing the system to determine the sentiment of tweets and identify meaningful named entities present within them.

Algorithm:

1. Import the required libraries and load the Sentiment140 CSV dataset with its six columns: target, ids, date, flag, user, and text.
2. Select a manageable sample of 50,000 tweets, remove duplicate or empty tweets, and retain only the sentiment and text information required for analysis.
3. Perform basic NLP preprocessing by converting tweets to lowercase and removing URLs, mentions, punctuation, numbers, special characters, and extra spaces.
4. Convert the cleaned tweets into numerical feature vectors using TF-IDF Vectorization with unigrams and bigrams.
5. Split the TF-IDF features into training and testing sets and train a Logistic Regression model for sentiment classification.
6. Predict the sentiment of test tweets and evaluate the model using accuracy, classification report, and confusion matrix.
7. Load the pretrained spaCy English model and process selected tweets to perform Named Entity Recognition.
8. Extract and display detected entities along with their corresponding labels, and demonstrate sentiment predictions on sample tweets.

Source Code:

```
!pip -q install spacy
!python -m spacy download en_core_web_sm -q
import pandas as pd
import numpy as np
import re
import string
import matplotlib.pyplot as plt
import seaborn as sns
import spacy
from google.colab import files
```

```
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    classification_report,
    confusion_matrix)
columns = [
    "target",
    "ids",
    "date",
    "flag",
    "user",
    "text"]
df = pd.read_csv(
    "/content/Train_Sentiment_Analysis_Dataset.csv",
    encoding="latin-1",
    names=columns)
print("\nDataset loaded successfully!")
print("Original Dataset Shape:", df.shape)
print("\nFirst 5 Records:")
print(df[["target", "text"]].head())
print("\nSentiment Distribution:")
print(df["target"].value_counts())
df = df.sample(
    n=50000,
    random_state=42).reset_index(drop=True)
df = df[df["target"].isin([0, 4])].copy()
df["label"] = df["target"].map({
    0: 0,
    4: 1})
df = df.dropna(subset=["text"])
df = df.drop_duplicates(subset=["text"])
print("\nSampled Dataset Shape:", df.shape)
def clean_text(text):
    text = str(text).lower()
    text = re.sub(r"http\S+|www\S+|https\S+", "", text)
    text = re.sub(r"@\w+", "", text)
    text = re.sub(r"\brt\b", "", text)
    text = text.translate(
        str.maketrans("", "", string.punctuation))
    text = re.sub(r"\d+", "", text)
    text = re.sub(r"\s+", " ", text).strip()
    return text
df["clean_text"] = df["text"].apply(clean_text)
print("\nOriginal Tweet:")
print(df["text"].iloc[0])
print("\nCleaned Tweet:")
```

```
print(df["clean_text"].iloc[0])
X_train, X_test, y_train, y_test = train_test_split(
    df["clean_text"],
    df["label"],
    test_size=0.20,
    random_state=42,
    stratify=df["label"])
print("\nTraining Samples:", len(X_train))
print("Testing Samples :", len(X_test))
vectorizer = TfidfVectorizer(
    max_features=10000,
    stop_words="english",
    ngram_range=(1, 2))
X_train_tfidf = vectorizer.fit_transform(X_train)
X_test_tfidf = vectorizer.transform(X_test)
print("\nTF-IDF Feature Matrix:")
print("Training:", X_train_tfidf.shape)
print("Testing :", X_test_tfidf.shape)
model = LogisticRegression(
    max_iter=1000)
model.fit(
    X_train_tfidf,
    y_train)
y_pred = model.predict(X_test_tfidf)
accuracy = accuracy_score(
    y_test,
    y_pred)
print("\n================================")
print("SENTIMENT ANALYSIS RESULTS")
print("================================")
print(f"Accuracy: {accuracy * 100:.2f}%")
print("\nClassification Report:")
print(
    classification_report(
        y_test,
        y_pred,
        target_names=[
            "Negative",
            "Positive"]))
cm = confusion_matrix(
    y_test,
    y_pred)
plt.figure(figsize=(6, 5))
sns.heatmap(
    cm,
    annot=True,
    fmt="d",
    cmap="Blues",
```

```
    xticklabels=[
        "Negative",
        "Positive"
    ],
    yticklabels=[
        "Negative",
        "Positive"])
plt.xlabel("Predicted Sentiment")
plt.ylabel("Actual Sentiment")
plt.title("Sentiment Analysis Confusion Matrix")
plt.show()
print("\n================================")
print("NAMED ENTITY RECOGNITION")
print("================================")
nlp = spacy.load("en_core_web_sm")
sample_tweets = df["text"].sample(
    5,
    random_state=10)
for i, tweet in enumerate(
    sample_tweets,
    start=1):
    doc = nlp(tweet)
    print(f"\nTweet {i}:")
    print(tweet)
    print("\nNamed Entities:")
    if doc.ents:
        for entity in doc.ents:
            print(
                f"{entity.text} --> {entity.label_}")
    else:
        print("No named entities detected.")
custom_tweets = [
    "I absolutely love this movie! It is amazing.",
    "This is the worst experience ever. I hate it.",
    "The new phone from Apple looks amazing!",
    "I visited New York and had a wonderful time."]
custom_clean = [
    clean_text(tweet)
    for tweet in custom_tweets]
custom_tfidf = vectorizer.transform(
    custom_clean)
custom_predictions = model.predict(
    custom_tfidf)
print("\n================================")
print("CUSTOM TWEET SENTIMENT")
print("================================")
for tweet, prediction in zip(
    custom_tweets,
```

```
    custom_predictions):
    sentiment = (
        "Positive"
        if prediction == 1
        else "Negative")
    print("\nTweet:", tweet)
    print("Predicted Sentiment:", sentiment)
print("\n==================================")
print("NER ON CUSTOM TWEETS")
print("==================================")
for tweet in custom_tweets:
    doc = nlp(tweet)
    print("\nTweet:", tweet)
    if doc.ents:
        for entity in doc.ents:
            print(
                f"Entity: {entity.text} | "
                f"Type: {entity.label_}")
    else:
        print("No named entities detected.")
print(
    f"Sentiment Analysis Accuracy: "
    f"{accuracy * 100:.2f}%")
```

Output:

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 12.8/12.8 MB 92.7 MB/s eta 0:00:00
✓ Download and installation successful
You can now load the package via spacy.load('en_core_web_sm')
⚠ Restart to reload dependencies
If you are in a Jupyter or Colab notebook, you may need to restart Python in
order to load all the package's dependencies. You can do this by selecting the
'Restart kernel' or 'Restart runtime' option.

Dataset loaded successfully!
Original Dataset Shape: (1600000, 6)

First 5 Records:
   target                                               text
0       0  @switchfoot http://twitpic.com/2y1zl - Awww, t...
1       0  is upset that he can't update his Facebook by ...
2       0  @Kenichan I dived many times for the ball. Man...
3       0    my whole body feels itchy and like its on fire
4       0  @nationwideclass no, it's not behaving at all....

Sentiment Distribution:
target
0    800000
4    800000
Name: count, dtype: int64

Sampled Dataset Shape: (49890, 7)

Original Tweet:
@chrishasboobs AHHH I HOPE YOUR OK!!!

Cleaned Tweet:
ahhh i hope your ok
```

```
Training Samples: 39912
Testing Samples : 9978

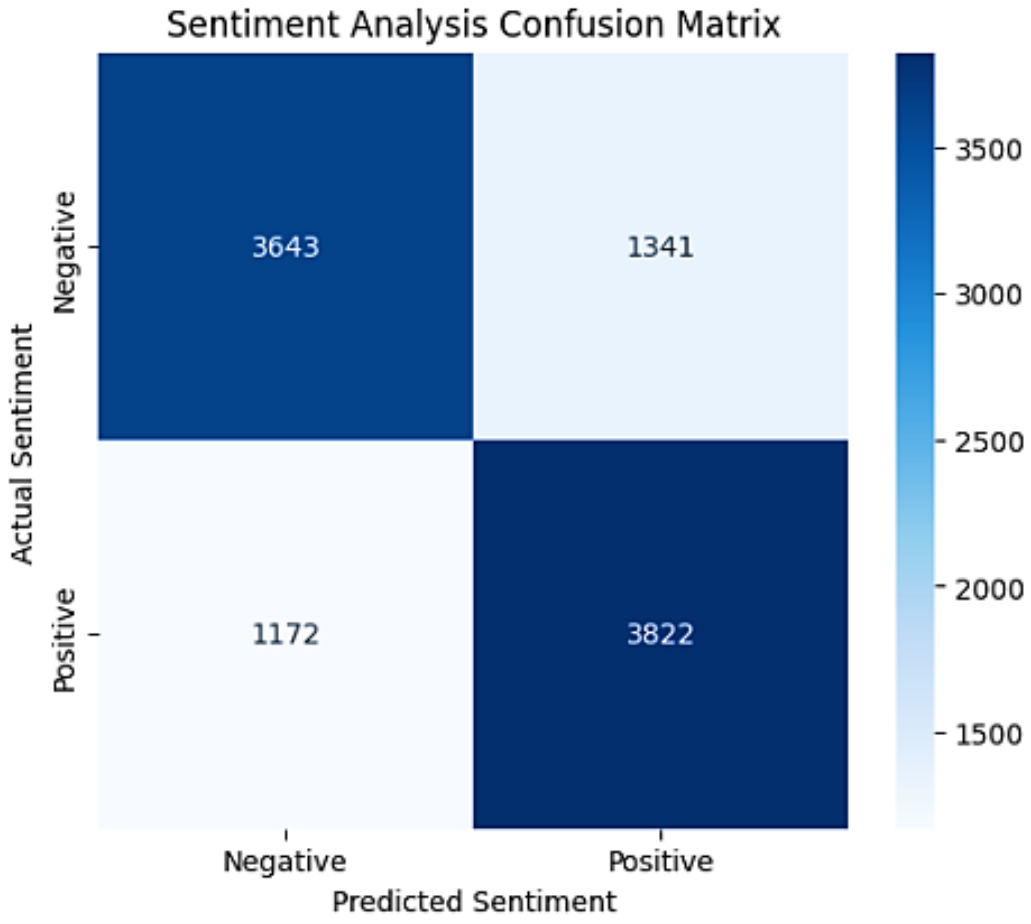
TF-IDF Feature Matrix:
Training: (39912, 10000)
Testing : (9978, 10000)

================================
SENTIMENT ANALYSIS RESULTS
================================
Accuracy: 74.81%

Classification Report:
              precision    recall  f1-score   support

    Negative       0.76      0.73      0.74      4984
    Positive       0.74      0.77      0.75      4994

    accuracy                           0.75      9978
   macro avg       0.75      0.75      0.75      9978
weighted avg       0.75      0.75      0.75      9978
```



```
================================
NAMED ENTITY RECOGNITION
================================

Tweet 1:
*_* freaky. yet sad.

Named Entities:
No named entities detected.

Tweet 2:
@chynnasaur hell yeaaah. why aren't you going to atl

Named Entities:
@chynnasaur hell yeaaah --> ORG

Tweet 3:
@TaLisaDuhhh you say fuck them w. a smiley face ? nooooo  lol

Named Entities:
No named entities detected.

Tweet 4:
@lamunro we can be as noisy as we like, twitter has told @hypnophil to come back in an hour - he's not a very happy twitterer

Named Entities:
@lamunro --> PERSON
@hypnophil --> PERSON

Tweet 5:
Crap, when my wallet was stolen last week, I lost my @Starbucks card! Bummer - losing out on discount!

Named Entities:
last week --> DATE
Bummer --> PERSON
```

```
================================
CUSTOM TWEET SENTIMENT
================================

Tweet: I absolutely love this movie! It is amazing.
Predicted Sentiment: Positive

Tweet: This is the worst experience ever. I hate it.
Predicted Sentiment: Negative

Tweet: The new phone from Apple looks amazing!
Predicted Sentiment: Positive

Tweet: I visited New York and had a wonderful time.
Predicted Sentiment: Positive

================================
NER ON CUSTOM TWEETS
================================

Tweet: I absolutely love this movie! It is amazing.
No named entities detected.

Tweet: This is the worst experience ever. I hate it.
No named entities detected.

Tweet: The new phone from Apple looks amazing!
Entity: Apple | Type: ORG

Tweet: I visited New York and had a wonderful time.
Entity: New York | Type: GPE
Sentiment Analysis Accuracy: 74.81%
```

Learning Outcome:

PRACTICAL NO. 5

Aim: Build RL agents and train them using OpenAI Gym or Stable Baselines to solve challenging tasks.

Theory:

Advanced Machine Learning and Reinforcement Learning: Advanced Machine Learning (AML) includes techniques that allow systems to learn complex patterns, make decisions, and improve their performance through experience. Reinforcement Learning (RL) is an important area of machine learning in which an agent learns by interacting with an environment. Unlike supervised learning, the agent is not given the correct action for every situation. Instead, it tries different actions and receives rewards or penalties from the environment. The objective is to learn a strategy that maximizes cumulative reward over time. RL is widely used in robotics, games, autonomous systems, resource management, and control problems. In this experiment, a PPO-based RL agent is trained to learn an appropriate policy for solving the CartPole environment.

Components of Reinforcement Learning: An RL system consists of several important components: the agent, environment, state, action, and reward. The agent is the decision-making system that learns from its experience. The environment represents the task in which the agent operates and provides feedback after every action. A state or observation describes the current condition of the environment and is used by the agent to select an action. An action represents the decision taken by the agent. A reward provides feedback about whether the selected action was useful or undesirable. The interaction can be represented as State → Action → Reward → New State → Learning. Through repeated interactions, the agent gradually improves its decision-making policy.

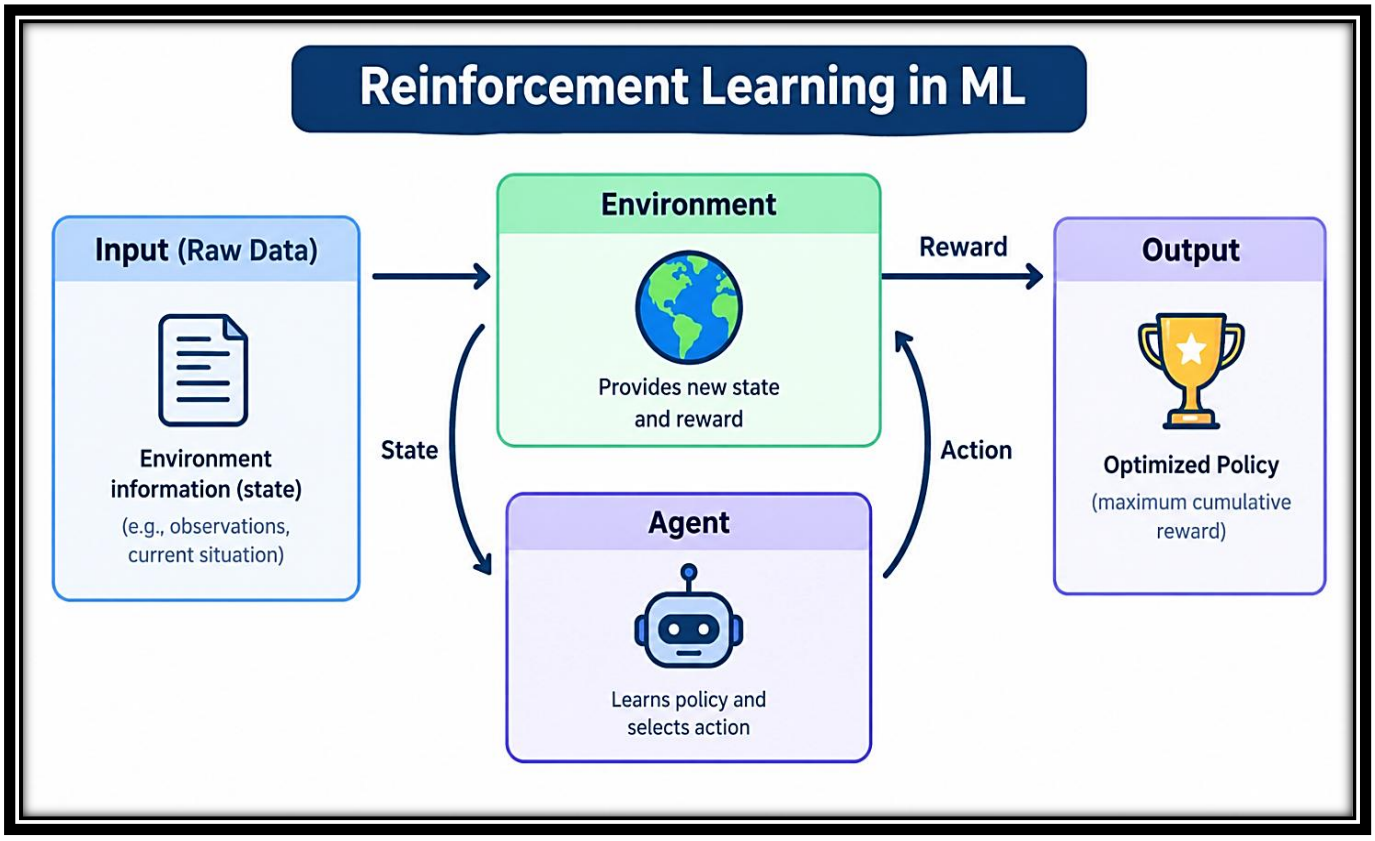
CartPole Environment: CartPole is a classic reinforcement learning environment in which a pole is attached to a moving cart. The objective is to control the cart so that the pole remains upright for as long as possible. The agent receives an observation containing information about the cart and pole, including cart position, cart velocity, pole angle, and pole angular velocity. The environment provides two possible actions: moving the cart to the left or moving it to the right. The agent receives rewards for maintaining the pole in a balanced position. An episode ends when the pole falls, the cart moves outside the allowed position, or the environment reaches its maximum number of steps. The agent therefore learns actions that help maintain the pole's balance.

Stable-Baselines3 and PPO: Stable-Baselines3 is a Python library that provides implementations of commonly used reinforcement learning algorithms. It allows RL agents to be trained without manually implementing the complete mathematical details of each algorithm. In this experiment, Proximal Policy Optimization (PPO) is used to train the agent. PPO is a policy-based reinforcement learning algorithm that improves the agent's policy using experiences collected through interaction with the environment. An important feature of PPO is that it limits excessively large policy updates, allowing the policy to improve gradually. The experiment uses an MLP policy, where a Multi-Layer Perceptron receives the environment's observations and helps determine the action selected by the agent.

RL Training and Performance Evaluation: During training, the RL agent repeatedly observes the environment, selects an action, receives a reward, and observes the resulting new state. This process continues for multiple timesteps and episodes, allowing the agent to learn from its experiences. The agent must balance exploration, where it tries different actions, and exploitation, where it uses actions that have previously produced good results. The performance of an RL agent can be evaluated using metrics such as average episode reward, reward standard deviation, episode length, and success rate when a clear success condition exists. In the CartPole experiment, the average episode reward and episode length can be used to determine whether the trained agent has learned to maintain the pole's balance.

Algorithm:

1. Install and import Gymnasium, Stable-Baselines3, PPO, NumPy, Pandas, and Matplotlib libraries.
2. Create the CartPole-v1 environment and initialize a PPO agent with an MLP policy.
3. Train the CartPole agent for 10,000 timesteps through repeated interaction with the environment.
4. Evaluate the trained CartPole agent for 10 episodes and calculate its average reward and reward standard deviation.
5. Create the LunarLander-v3 environment and initialize another PPO agent with an MLP policy.
6. Train the LunarLander agent for 200,000 timesteps and evaluate it over 10 episodes.
7. Store the results of both agents, including environment, algorithm, training steps, average reward, and reward standard deviation, in a results table.
8. Plot the average episode rewards of the two trained agents and display the final training results.



Source Code:

```
!pip -q install "stable-baselines3[extra]" "gymnasium[box2d]"
import gymnasium as gym
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from stable_baselines3 import PPO
from stable_baselines3.common.evaluation import
evaluate_policy
from IPython.display import display

print("TRAINING CARTPOLE AGENT...")
print("The agent learns to keep the pole balanced.")

cartpole_env = gym.make("CartPole-v1")
print("\nObservation Space:", cartpole_env.observation_space)
print("Action Space:", cartpole_env.action_space)

cartpole_model = PPO(
    "MlpPolicy",
    cartpole_env,
    verbose=0,
    seed=42)

cartpole_model.learn(total_timesteps=10000)

cartpole_mean_reward, cartpole_std_reward = evaluate_policy(
    cartpole_model,
    cartpole_env,
    n_eval_episodes=10,
    deterministic=True)

cartpole_env.close()

print("\nCartPole training completed.")

print("Average Reward:", round(cartpole_mean_reward, 2))
print("Reward Std.:", round(cartpole_std_reward, 2))

print("TRAINING LUNARLANDER AGENT")
print("The agent learns to land the spacecraft safely.")

lander_env = gym.make("LunarLander-v3")
print("\nObservation Space:", lander_env.observation_space)
print("Action Space:", lander_env.action_space)
lander_model = PPO(
    "MlpPolicy",
    lander_env,
    verbose=0,
    seed=42)
```

```
lander_model.learn(
    total_timesteps=200000)
lander_mean_reward, lander_std_reward = evaluate_policy(
    lander_model,
    lander_env,
    n_eval_episodes=10,
    deterministic=True)
lander_env.close()

print("\nLunarLander training completed.")

print("Average Reward:", round(lander_mean_reward, 2))
print("Reward Std.:", round(lander_std_reward, 2))
results = pd.DataFrame({
    "Environment": [
        "CartPole-v1",
        "LunarLander-v3"],
    "Algorithm": [
        "PPO",
        "PPO"],
    "Training Steps": [
        10000,
        200000],
    "Average Reward": [
        round(cartpole_mean_reward, 2),
        round(lander_mean_reward, 2)],
    "Reward Std.": [
        round(cartpole_std_reward, 2),
        round(lander_std_reward, 2)]})

print("REINFORCEMENT LEARNING RESULTS")
display(results)
plt.figure(figsize=(8, 5))

plt.bar(results["Environment"],
    results["Average Reward"])

plt.title("Performance of Trained RL Agents")
plt.xlabel("Environment")
plt.ylabel("Average Episode Reward")
plt.grid(axis="y",alpha=0.3)
plt.show()

print(f"\nCartPole Average Reward: "
    f"{cartpole_mean_reward:.2f}")
print(f"LunarLander Average Reward: "
    f"{lander_mean_reward:.2f}")
print("\nHigher average reward indicates better "
    "performance within the respective environment.")
```

Output:

```
TRAINING CARTPOLE AGENT...
The agent learns to keep the pole balanced.

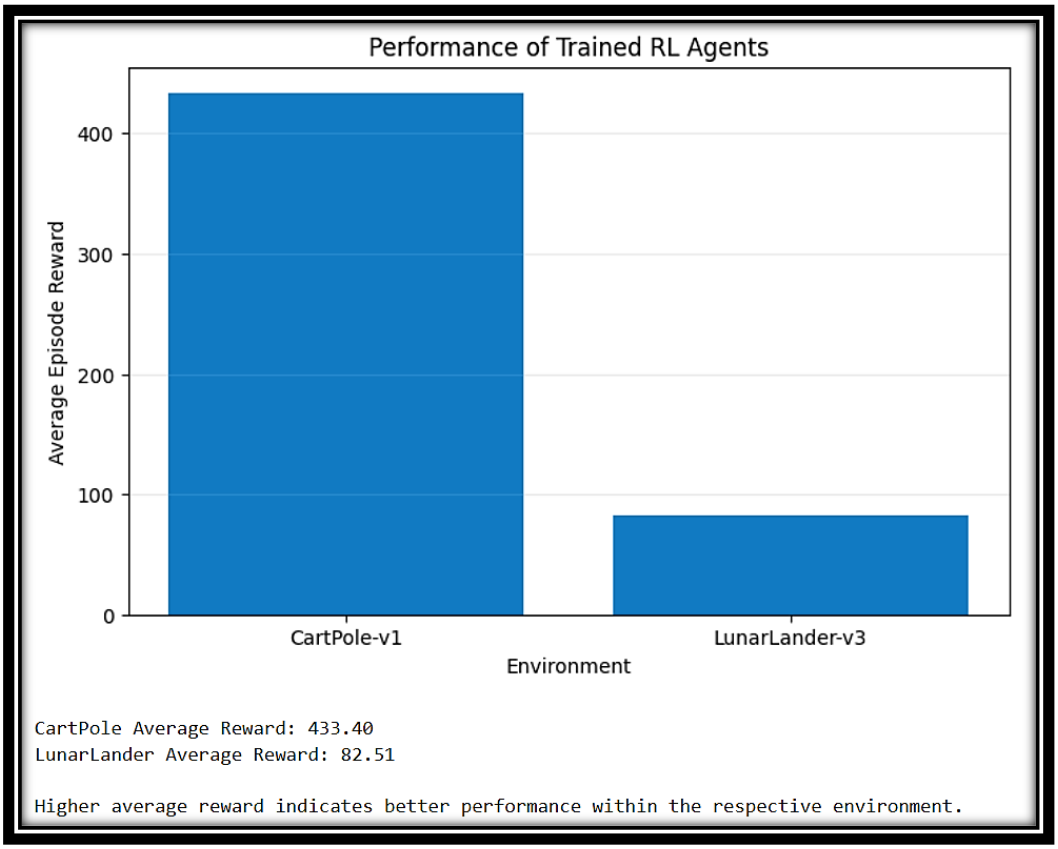
Observation Space: Box([-4.8               -inf -0.41887903        -inf], [4.8               inf 0.41887903        inf], (4,), float32)
Action Space: Discrete(2)
/usr/local/lib/python3.13/dist-packages/stable_baselines3/common/evaluation.py:71: UserWarning: Evaluation environment is not wrapped with a ``Monitor`` wrapper. This may result in reporting modified episode lengths and
  warnings.warn(

CartPole training completed.
Average Reward: 433.4
Reward Std.: 82.26
TRAINING LUNARLANDER AGENT
The agent learns to land the spacecraft safely.

Observation Space: Box([ -2.5        -2.5       -10.        -10.        -6.2831855 -10.
  -0.         -0.        ], [ 2.5        2.5       10.        10.        6.2831855 10.
  1.         1.        ], (8,), float32)
Action Space: Discrete(4)

LunarLander training completed.
Average Reward: 82.51
Reward Std.: 128.6
REINFORCEMENT LEARNING RESULTS
/usr/local/lib/python3.13/dist-packages/jupyter_client/session.py:203: DeprecationWarning: datetime.datetime.utcnow() is deprecated and scheduled for removal in a future version. Use timezone-aware objects to represent
  return datetime.utcnow().replace(tzinfo=utc)
```

	Environment	Algorithm	Training Steps	Average Reward	Reward Std.
0	CartPole-v1	PPO	10000	433.40	82.26
1	LunarLander-v3	PPO	200000	82.51	128.60



Learning Outcome:

PRACTICAL NO. 6

Aim: Understand the interpretability of ML models by using LIME or SHAP to explain model predictions.

Theory:

Machine Learning Model Interpretability: Machine learning model interpretability refers to understanding how a model makes predictions and which features influence its decisions. Although machine learning models can achieve high accuracy, understanding their decision-making process is important for transparency, reliability, and trust. Complex models such as Random Forests combine multiple decision trees, making individual predictions difficult to interpret directly. Explainable Artificial Intelligence (XAI) provides techniques to investigate these predictions and identify influential features. In this experiment, LIME and SHAP are used to explain the predictions of a trained Random Forest classifier. These techniques help users understand model behaviour rather than treating its predictions as unexplained outputs.

Dataset Description: The Breast Cancer Wisconsin dataset, available through Scikit-learn, contains 569 samples and 30 numerical features extracted from digitized images of fine needle aspirates of breast masses. These features describe cell-nucleus characteristics such as radius, texture, perimeter, area, smoothness, compactness, concavity, symmetry, and fractal dimension. The target variable contains two classes: malignant and benign. This dataset is commonly used for binary classification and evaluating machine learning models. In this experiment, the dataset is divided into training and testing sets using an 80:20 ratio. A Random Forest classifier is trained to predict the target class, after which LIME and SHAP are applied to investigate the features influencing the model's predictions.

LIME (Local Interpretable Model-Agnostic Explanations): LIME is an explainable AI technique that explains an individual prediction by approximating a complex model's behavior around a particular input using a simpler, interpretable model. It generates perturbed samples around the selected instance, obtains predictions from the original model, and assigns weights based on the similarity of those samples to the original instance. A simple local model is then fitted to these weighted samples to identify influential features. In this experiment, LimeTabularExplainer is initialized using the training data and feature names. The explain_instance() method explains a selected test sample using the Random Forest model's predicted probabilities. The resulting explanation highlights features that influence the local prediction.

SHAP (SHapley Additive Explanations): SHAP is an explainable AI technique based on Shapley values from cooperative game theory. It estimates how individual features contribute to a model's output relative to a baseline prediction. SHAP can explain individual predictions and summarize overall model behaviour. Positive and negative contributions indicate changes in the explained output in different directions. In this experiment, shap.TreeExplainer is used because Random Forest is a tree-based model. SHAP values are calculated for 100 test samples, and a summary bar plot displays global feature importance. Features with larger mean absolute SHAP values have a greater average impact on the model output across the selected samples.

Comparison and Applications: LIME and SHAP improve the interpretability of machine learning models in different ways. LIME focuses on explaining an individual prediction through a locally fitted interpretable model, whereas SHAP estimates feature contributions using Shapley values. Both techniques can help investigate unexpected predictions, identify influential features, and improve transparency in applications such as healthcare research, finance, and fraud detection. In this experiment, LIME explains one selected test instance, while SHAP provides an overview of feature importance across multiple samples. These explanations describe the behaviour of the trained model and do not establish causal relationships. Model accuracy and feature explanations alone are not sufficient to justify clinical decisions.

Algorithm:

1. Import the required libraries, including NumPy, Pandas, Matplotlib, Scikit-learn, LIME, and SHAP.
2. Load the built-in Breast Cancer Wisconsin dataset and separate the input features and target labels.
3. Display the dataset shape, feature names, target classes, and first five records.
4. Split the dataset into training and testing sets using an 80:20 ratio with stratification.
5. Initialize a Random Forest classifier with 100 trees and train it on the training data.
6. Predict the test labels and evaluate the model using accuracy and the classification report.
7. Initialize LimeTabularExplainer using the training data and explain one test prediction using explain_instance().
8. Display the actual and predicted classes and visualize the LIME feature contributions.
9. Initialize shap.TreeExplainer for the trained Random Forest model and calculate SHAP values for 100 test samples.
10. Generate a SHAP summary bar plot to display global feature importance and complete the experiment.

Source Code:

```
!pip -q install lime shap
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import shap
from IPython.display import display
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report
from lime.lime_tabular import LimeTabularExplainer

data = load_breast_cancer()
X = pd.DataFrame(data.data, columns=data.feature_names)
y = data.target
```

```
print("Dataset Shape:", X.shape)
print("Feature Names:", list(X.columns))
print("Target Classes:", list(data.target_names))
print("\nFirst Five Records:")
display(X.head())

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

model = RandomForestClassifier(
    n_estimators=100, random_state=42)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nModel Accuracy:", round(accuracy_score(y_test, y_pred), 4))
print("\nClassification Report:")
print(classification_report(
    y_test, y_pred, target_names=data.target_names))

lime_explainer = LimeTabularExplainer(
    training_data=X_train.to_numpy(),
    feature_names=list(X.columns),
    class_names=list(data.target_names),
    mode="classification",
    random_state=42)

instance_index = 0
instance = X_test.iloc[instance_index].to_numpy()
```

```
lime_exp = lime_explainer.explain_instance(
    data_row=instance,
    predict_fn=lambda samples: model.predict_proba(
        pd.DataFrame(samples, columns=X.columns)),
    num_features=10,
    num_samples=3000)

actual_class = data.target_names[y_test[instance_index]]
predicted_class = data.target_names[
    model.predict(X_test.iloc[[instance_index]])[0]]

print("\nLIME EXPLANATION")
print("Actual Class:", actual_class)
print("Predicted Class:", predicted_class)

display(lime_exp.show_in_notebook(show_table=True))
fig = lime_exp.as_pyplot_figure()
plt.tight_layout()
plt.show()
shap_explainer = shap.TreeExplainer(model)
X_explain = X_test.iloc[:100]
shap_values = shap_explainer.shap_values(X_explain)

if isinstance(shap_values, list):
    class_shap_values = shap_values[0]
elif isinstance(shap_values, np.ndarray) and shap_values.ndim == 3:
    class_shap_values = shap_values[:, :, 0]
else:
    class_shap_values = shap_values

print("\nSHAP Values Shape:", class_shap_values.shape)

shap.summary_plot(
    class_shap_values,
    X_explain,
    plot_type="bar",
    show=False)
plt.title("SHAP Feature Importance")
plt.tight_layout()
plt.show()
```

Output:

```
Dataset Shape: (569, 30)
Feature Names: ['mean radius', 'mean texture', 'mean perimeter', 'mean area', 'mean smoothness', 'mean compactness', 'mean concavity', 'mean concave points', 'mean symmetry', 'mean fractal dimension', 'radius error', 'texture error', 'perimeter error
Target Classes: [np.str_('malignant'), np.str_('benign')]

First Five Records:
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness	worst concavity	worst concave points	worst symmetry	worst fractal dimension
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.6656	0.7119	0.2654	0.4601	0.11890
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	0.1866	0.2416	0.1860	0.2750	0.08902
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	0.4245	0.4504	0.2430	0.3613	0.08758
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098	0.8663	0.6869	0.2575	0.6638	0.17300
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	0.2050	0.4000	0.1625	0.2364	0.07678

```
5 rows × 30 columns

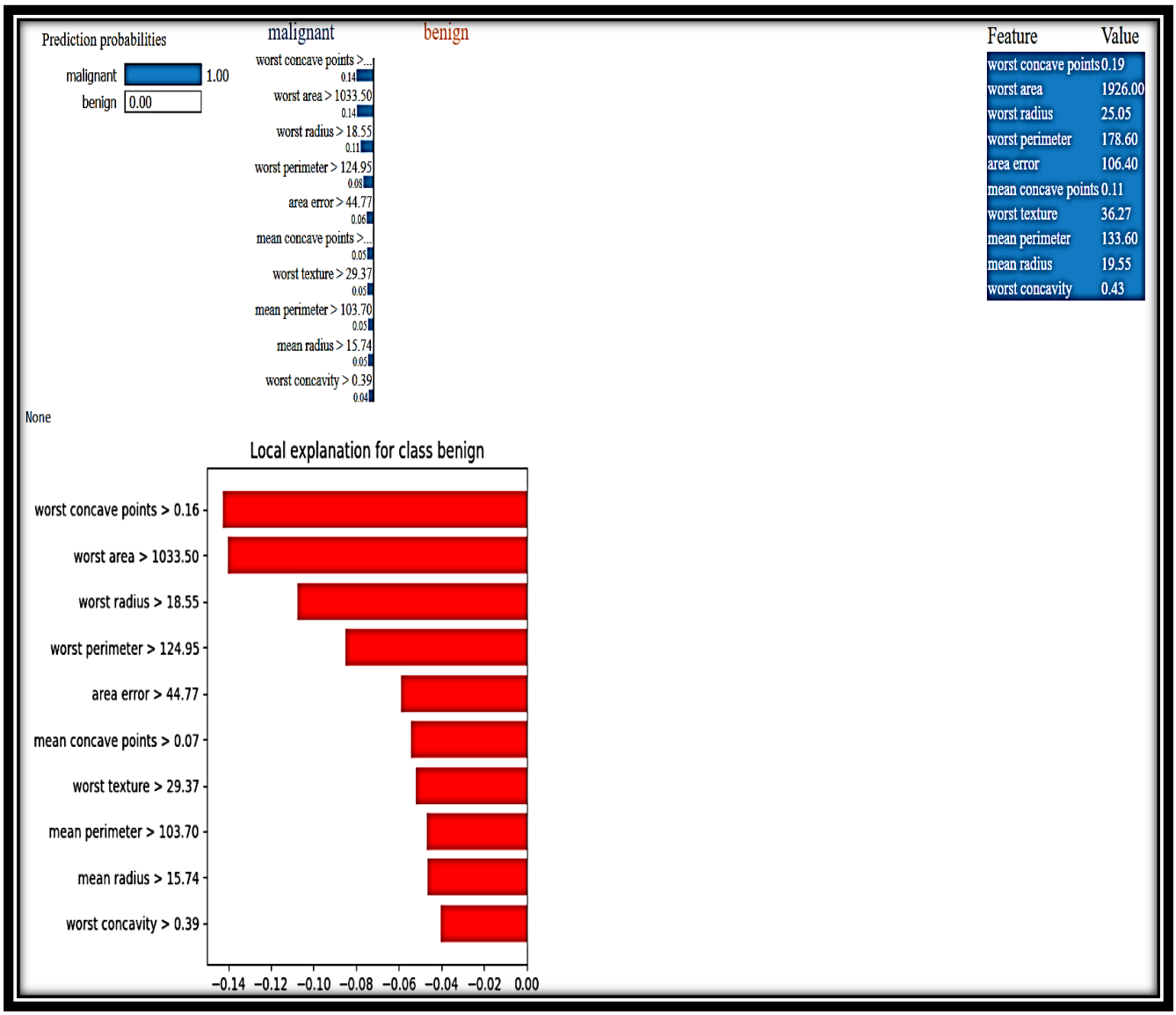
Model Accuracy: 0.9561

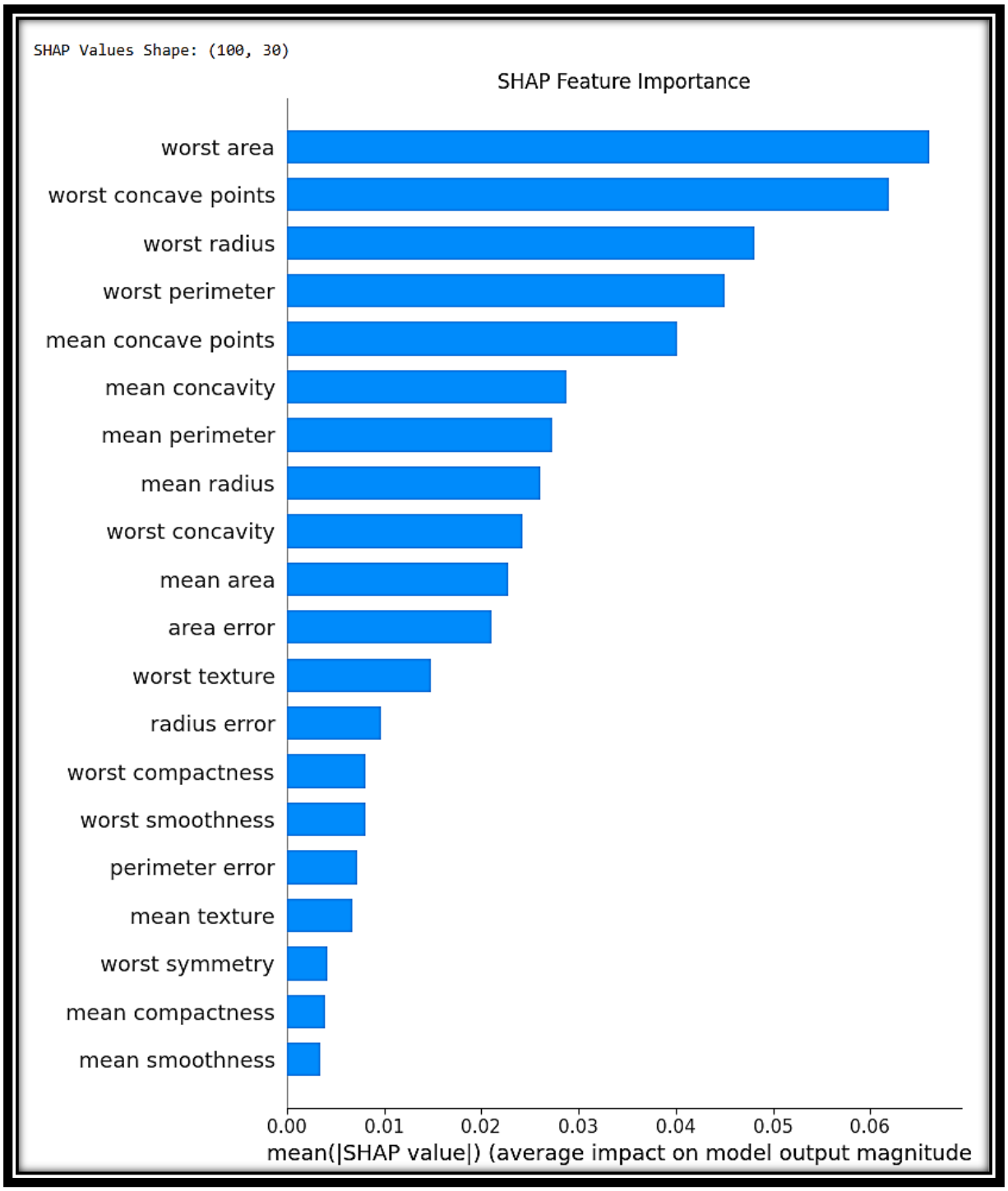
Classification Report:
              precision    recall  f1-score   support

   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

    accuracy                           0.96       114
   macro avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114


LIME EXPLANATION
Actual Class: malignant
Predicted Class: malignant
```





Learning Outcome:

PRACTICAL NO. 7

Aim: Use AutoML and Hyperparameter tuning tools to automate the model selection and optimization process.

Theory:

Automated Machine Learning (AutoML): Automated Machine Learning (AutoML) is a technique used to automate important stages of developing a machine learning model. Normally, building a model involves selecting an algorithm, preparing data, choosing suitable parameters, training the model, and evaluating its performance. These steps can require considerable time and experimentation. AutoML reduces manual effort by automatically exploring possible configurations and identifying models that perform well according to a selected evaluation metric. It can assist with model selection, feature processing, hyperparameter optimization, and performance evaluation. AutoML is useful when several configurations need to be tested systematically. In this experiment, automated tuning is used to improve a neural network's performance on handwritten digit classification.

Hyperparameters and Their Importance: Hyperparameters are settings selected before or during the configuration of model training, rather than values directly learned as ordinary model weights. Examples include the number of neurons in a neural network, the learning rate, the number of training epochs, batch size, and dropout rate. These settings influence how quickly a model learns, its ability to generalize to unseen data, and the possibility of overfitting. An unsuitable learning rate may prevent effective learning, while an unnecessarily complex network may increase computational cost. Hyperparameter tuning systematically tests different combinations of these settings and compares their validation performance. The best configuration is selected according to a predefined objective, such as maximizing validation accuracy or minimizing validation loss.

Hyperparameter Tuning Techniques: Several methods are available for hyperparameter optimization. Grid Search evaluates every combination in a predefined parameter grid, but it can become computationally expensive when many settings are involved. Random Search samples a limited number of configurations from specified ranges or candidate values, allowing a wider search with a controlled computational budget. Bayesian optimization uses information from earlier trials to guide subsequent experiments toward promising configurations. Tools such as Keras Tuner support methods including Random Search and Bayesian Optimization. In this experiment, Random Search is used to test a small number of neural-network configurations. Each trial uses different hyperparameter values, and the configurations are compared using validation accuracy to identify a promising model.

Keras Tuner and Neural Network Optimization: Keras Tuner is a Python library designed to simplify hyperparameter optimization for TensorFlow and Keras models. It allows developers to define a model-building function containing tunable parameters and then automatically search for suitable configurations. In this experiment, a feedforward neural network is built using a dense hidden layer, ReLU activation, an optional dropout layer, and a softmax output layer. The number of hidden neurons, dropout rate, and learning rate are selected as tunable hyperparameters. The model is trained using the Adam optimizer and sparse categorical cross-entropy loss. Keras Tuner's Random Search trains the candidate

configurations and selects the best trial based on validation accuracy, reducing the need to configure and compare every model manually.

Applications and Evaluation: AutoML and hyperparameter tuning are useful in applications such as image classification, medical diagnosis, sales prediction, fraud detection, and customer analytics. They help practitioners discover effective configurations while reducing repetitive manual experimentation. In this experiment, the MNIST dataset is used to classify handwritten digits from zero to nine. The dataset is divided into training and testing sets, while a validation portion of the training data is used during tuning. The best configuration is subsequently evaluated on the separate test dataset. Test accuracy measures the proportion of correctly classified images, while a comparison of trial validation scores illustrates how different configurations perform. The test set is reserved for final evaluation rather than selecting hyperparameters.

Dataset Description:

The MNIST (Modified National Institute of Standards and Technology) dataset is a standard benchmark dataset available directly through TensorFlow and Keras. It contains 70,000 grayscale images of handwritten digits ranging from 0 to 9, each having a resolution of 28 × 28 pixels. The dataset consists of 60,000 training images and 10,000 testing images. Each image contains pixel intensity values representing a handwritten digit, and its corresponding label indicates the correct digit class. In this experiment, the images are flattened into 784 numerical features and normalized to the range 0–1. A feedforward neural network learns to classify the digits. Keras Tuner then searches for suitable hyperparameters by comparing candidate models using validation accuracy.

Algorithm:

1. Import TensorFlow, Keras Tuner, NumPy, Pandas, and Matplotlib libraries.
2. Load the MNIST dataset, separate the training and testing sets, flatten each image into 784 features, and normalize pixel values to the range 0–1.
3. Define a feedforward neural network with a tunable number of hidden neurons, dropout rate, and learning rate, using ReLU and softmax activation functions.
4. Compile the model using the Adam optimizer, sparse categorical cross-entropy loss, and accuracy as the evaluation metric.
5. Initialize Keras Tuner with Random Search, specifying validation accuracy as the optimization objective and a maximum of five trials.
6. Train the candidate models for three epochs each, using 20% of the training data for validation and a batch size of 128.
7. Retrieve and display the best hyperparameter configuration identified by the tuning process.
8. Evaluate the best model on the separate test dataset and display its test loss and accuracy.
9. Collect the validation accuracy and hyperparameter values of the completed trials and display them in a results table.
10. Plot the validation accuracy of the tuning trials using a bar chart.
11. Generate predictions for ten test images and display each image with its actual and predicted digit labels.
12. Display the completion message and end the experiment.

Source Code:

```
!pip -q install keras-tuner
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
import keras_tuner as kt
print("TensorFlow Version:", tf.__version__)
(x_train, y_train), (x_test, y_test) = (
    tf.keras.datasets.mnist.load_data())

x_train = x_train.reshape(-1, 784).astype("float32") / 255.0
x_test = x_test.reshape(-1, 784).astype("float32") / 255.0
print("\nMNIST Dataset")
print("Training images:", x_train.shape)
print("Testing images :", x_test.shape)

def build_model(hp):
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(784,)),
        tf.keras.layers.Dense(
            units=hp.Int("units", min_value=32,
                         max_value=256, step=32),
            activation="relu"),
        tf.keras.layers.Dropout(
            rate=hp.Float("dropout", min_value=0.0,
                          max_value=0.5, step=0.1)),
        tf.keras.layers.Dense(10, activation="softmax")])
    model.compile(optimizer=tf.keras.optimizers.Adam(
            learning_rate=hp.Float(
                "learning_rate", min_value=1e-4,
                max_value=1e-2, sampling="log")),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])
    return model
```

```
tuner = kt.RandomSearch(
    hypermodel=build_model,
    objective="val_accuracy",
    max_trials=5,
    executions_per_trial=1,
    directory="automl_tuning",
    project_name="mnist_optimization",
    overwrite=True,
    seed=42)

print("\nStarting Hyperparameter Tuning...")
tuner.search(
    x_train, y_train,
    epochs=3,
    validation_split=0.2,
    batch_size=128,
    verbose=1)

best_hp = tuner.get_best_hyperparameters(num_trials=1)[0]
print("\nBest Hyperparameters:")
print("Hidden Neurons:", best_hp.get("units"))
print("Dropout Rate  :", best_hp.get("dropout"))
print("Learning Rate :", best_hp.get("learning_rate"))

best_model = tuner.get_best_models(num_models=1)[0]

test_loss, test_accuracy = best_model.evaluate(
    x_test, y_test, verbose=0)

print("\nBest Model Test Results")
print("Test Loss     :", round(test_loss, 4))
print("Test Accuracy:", round(test_accuracy * 100, 2), "%")
```

```
trials = tuner.oracle.get_best_trials(num_trials=5)

results = pd.DataFrame([
    {
        "Trial": f"Trial {i + 1}",
        "Hidden Neurons": trial.hyperparameters.get("units"),
        "Dropout Rate": trial.hyperparameters.get("dropout"),
        "Learning Rate": trial.hyperparameters.get("learning_rate"),
        "Validation Accuracy": trial.score
    }
    for i, trial in enumerate(trials)
    if trial.score is not None])

print("\nHyperparameter Tuning Results")
display(results.round(4))

plt.figure(figsize=(8, 5))
plt.bar(results["Trial"],
    results["Validation Accuracy"])
plt.title("Hyperparameter Tuning Results")
plt.xlabel("Trial")
plt.ylabel("Validation Accuracy")
plt.ylim(0, 1.05)
plt.grid(axis="y", alpha=0.3)
plt.tight_layout()
plt.show()

predictions = np.argmax(
    best_model.predict(x_test[:10], verbose=0), axis=1)

plt.figure(figsize=(10, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].reshape(28, 28), cmap="gray")
    plt.title(f"Actual: {y_test[i]}\nPredicted: {predictions[i]}")
    plt.axis("off")

plt.suptitle("MNIST Predictions Using the Tuned Model")
plt.tight_layout()
plt.show()
```

Output:

```
Trial 5 Complete [00h 00m 10s]
val_accuracy: 0.9649166464805603

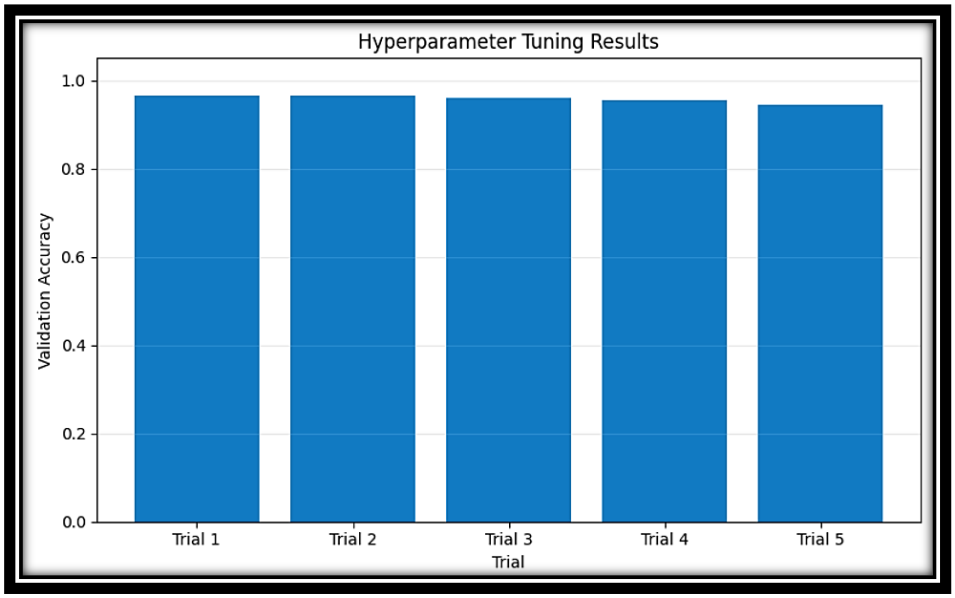
Best val_accuracy So Far: 0.9649166464805603
Total elapsed time: 00h 00m 43s

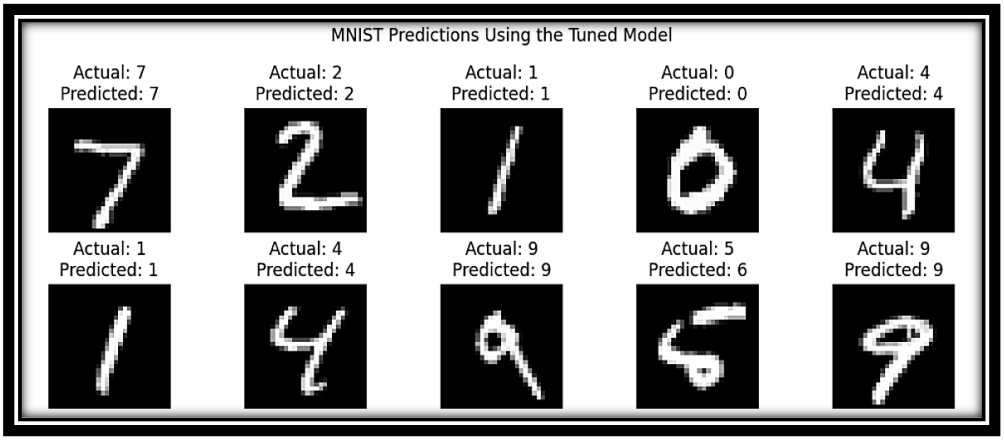
Best Hyperparameters:
Hidden Neurons: 160
Dropout Rate  : 0.0
Learning Rate : 0.0009887195863122636
/usr/local/lib/python3.13/dist-packages/keras/src/saving/saving_lib.py:797: UserWarning: Skipping variable loading for optimizer 'adam', because it has 2 variables whereas the saved optimizer has 10 variables.
  saveable.load_own_variables(weights_store.get(inner_path))

Best Model Test Results
Test Loss     : 0.1192
Test Accuracy: 96.5 %

Hyperparameter Tuning Results
```

	Trial	Hidden Neurons	Dropout Rate	Learning Rate	Validation Accuracy
0	Trial 1	160	0.0	0.0010	0.9649
1	Trial 2	256	0.0	0.0085	0.9649
2	Trial 3	192	0.0	0.0007	0.9607
3	Trial 4	64	0.4	0.0017	0.9559
4	Trial 5	96	0.4	0.0005	0.9442





Learning Outcome:

PRACTICAL NO. 8

Aim: Analyse time series data, perform forecasting, and evaluate model performance using Prophet or stats-models.

Theory:

Time Series Analysis: Time series analysis is a statistical technique used to study observations collected over time at regular intervals, such as daily, monthly, quarterly, or yearly. Unlike ordinary datasets, time series data contains a chronological order that must be preserved during analysis. It helps identify important patterns such as trends, seasonality, and random fluctuations. A trend represents the long-term increase or decreases in values, while seasonality refers to patterns that repeat at fixed intervals. Time series analysis is widely used in business forecasting, stock market analysis, weather prediction, sales estimation, electricity demand planning, and passenger traffic prediction.

Time Series Forecasting: Time series forecasting is the process of predicting future values using historical observations. It assumes that useful information about future behaviour can be obtained by analysing past patterns. Forecasting involves collecting historical data, preprocessing it, visualising trends, selecting a suitable model, training the model, and generating predictions. The dataset is generally divided chronologically into training and testing sets, with earlier observations used for training and later observations reserved for evaluation. Common forecasting techniques include ARIMA, SARIMA, Exponential Smoothing, and Prophet. Forecasting helps organisations plan resources, estimate future demand, identify seasonal changes, and make informed decisions.

Statsmodels: Statsmodels is an open-source Python library used for statistical modelling, hypothesis testing, and time series analysis. It provides several forecasting methods, including ARIMA, Seasonal ARIMA (SARIMA), and Exponential Smoothing. These methods analyse historical observations to identify relationships, trends, and seasonal patterns that can be used to predict future values. In this experiment, the SARIMAX model from Statsmodels is used to forecast monthly airline passenger numbers. SARIMAX extends ARIMA by supporting seasonal patterns and optional external variables. The model learns from past passenger counts and generates forecasts along with confidence intervals, helping estimate both expected values and their uncertainty.

SARIMA Model: SARIMA stands for Seasonal AutoRegressive Integrated Moving Average. It is an extension of ARIMA designed for time series data containing repeating seasonal patterns. The model is represented by the parameters $(p,d,q)(P,D,Q,s)$, where p represents the autoregressive order, d represents the degree of differencing, and q represents the moving-average order. The seasonal parameters P,D,Q represent their seasonal counterparts, while s specifies the seasonal period. In this experiment, the parameters $(1,1,1)(1,1,1,12)$ are used because the data is monthly and exhibits a yearly seasonal pattern. Differencing helps reduce trends, while seasonal components capture recurring annual changes in passenger traffic.

Model Evaluation and Applications: A forecasting model must be evaluated by comparing its predictions with actual observations that were not used during training. Two common evaluation metrics are Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE).

MAE calculates the average absolute difference between actual and predicted values, while RMSE gives greater weight to larger prediction errors. Lower values generally indicate better forecasting accuracy when comparing models on the same dataset and scale. Forecast plots and confidence intervals provide additional insight into prediction quality and uncertainty. Time series forecasting is useful for airline passenger planning, inventory management, financial analysis, workforce allocation, and demand prediction. However, model performance depends on data quality, seasonal patterns, and the selected model parameters.

Dataset Description:

The Airline Passengers dataset is a commonly used time series dataset containing monthly totals of international airline passengers from January 1949 to December 1960. It contains 144 observations, with two columns: Month, representing the observation date, and Passengers, representing the number of passengers in thousands. The dataset demonstrates a clear upward trend and recurring yearly seasonal variations, making it suitable for learning time series analysis and forecasting. In this experiment, the first 132 observations are used to train the SARIMA model, while the final 12 observations are reserved for testing.

Algorithm:

1. Import Pandas, NumPy, Matplotlib, Statsmodels, and Scikit-learn libraries.
2. Load the monthly Airline Passengers dataset and display its first five records, dimensions, and missing values.
3. Convert the month column into datetime format and set it as the time-series index with monthly frequency.
4. Visualise historical passenger counts to identify trends and seasonal patterns.
5. Split the dataset chronologically into training data containing the first 132 observations and testing data containing the last 12 observations.
6. Initialise the SARIMAX model using non-seasonal parameters \((1,1,1)\) and seasonal parameters \((1,1,1,12)\).
7. Train the model using the training dataset and obtain its fitted results.
8. Forecast passenger counts for the 12-month testing period and extract the corresponding confidence intervals.
9. Compare the actual and predicted passenger counts in a table.
10. Calculate MAE and RMSE to evaluate forecasting performance.
11. Plot the training data, actual test values, predicted values, and confidence intervals.
12. Display diagnostic plots to examine the model residuals.
13. Interpret the forecasting results and end the experiment.

Source Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.statespace.sarimax import SARIMAX
from sklearn.metrics import mean_absolute_error, mean_squared_error

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"
data = pd.read_csv(url)
print("First Five Records:")
print(data.head())
print("\nDataset Shape:", data.shape)
```

```
data["Month"] = pd.to_datetime(data["Month"])
data = data.set_index("Month").asfreq("MS")

print("\nMissing Values:")
print(data.isnull().sum())

plt.figure(figsize=(12, 5))
plt.plot(data.index, data["Passengers"], marker="o")
plt.title("Monthly Airline Passenger Data")
plt.xlabel("Year")
plt.ylabel("Passengers (Thousands)")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()

train = data.iloc[:-12]
test = data.iloc[-12:]

print("Training Samples:", len(train))
print("Testing Samples:", len(test))

model = SARIMAX(
    train["Passengers"],
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    enforce_stationarity=False,
    enforce_invertibility=False)

results = model.fit(disp=False)

print("\nSARIMA Model Summary:")
print(results.summary())

forecast_result = results.get_forecast(steps=len(test))
predicted = forecast_result.predicted_mean
confidence = forecast_result.conf_int()
```

```
comparison = pd.DataFrame({
    "Actual": test["Passengers"].values,
    "Predicted": predicted.values,
    "Lower Bound": confidence.iloc[:, 0].values,
    "Upper Bound": confidence.iloc[:, 1].values}, index=test.index)

print("\nActual vs Predicted Passenger Counts:")
print(comparison.round(2))

mae = mean_absolute_error(
    comparison["Actual"], comparison["Predicted"])
rmse = np.sqrt(mean_squared_error(
    comparison["Actual"], comparison["Predicted"]))

print("\nModel Evaluation:")
print("Mean Absolute Error (MAE):", round(mae, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))

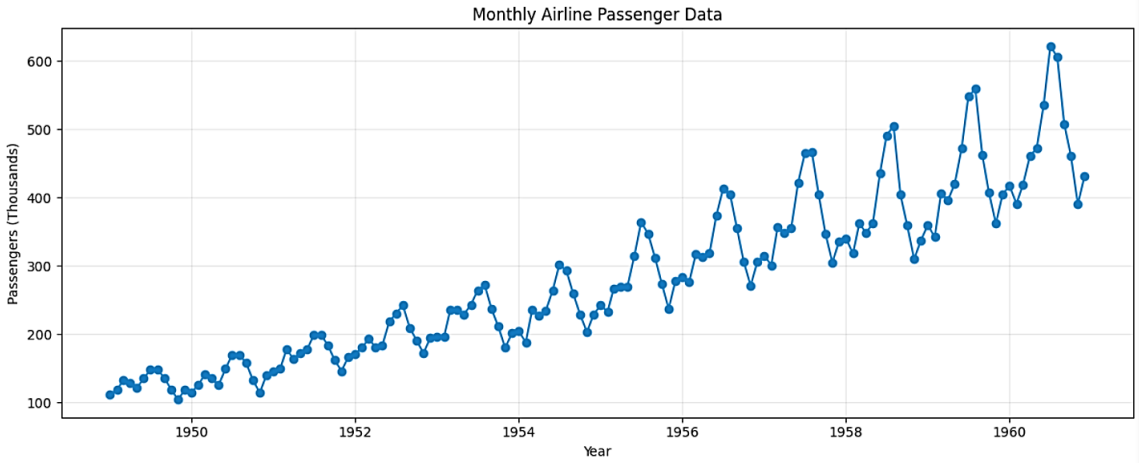
plt.figure(figsize=(12, 5))
plt.plot(train.index, train["Passengers"], label="Training Data")
plt.plot(test.index, test["Passengers"], marker="o", label="Actual Values")
plt.plot(test.index, predicted.values, marker="o", linestyle="--", label="Forecasted Values")
plt.fill_between(
    test.index,
    confidence.iloc[:, 0].values,
    confidence.iloc[:, 1].values,
    alpha=0.2,
    label="95% Confidence Interval")
plt.title("SARIMA Forecast: Actual vs Predicted Passengers")
plt.xlabel("Year")
plt.ylabel("Passengers (Thousands)")
plt.legend()
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
results.plot_diagnostics(figsize=(12, 8))
plt.tight_layout()
plt.show()
```

Output:

```
First Five Records:
     Month  Passengers
0  1949-01         112
1  1949-02         118
2  1949-03         132
3  1949-04         129
4  1949-05         121

Dataset Shape: (144, 2)

Missing Values:
Passengers    0
dtype: int64
```



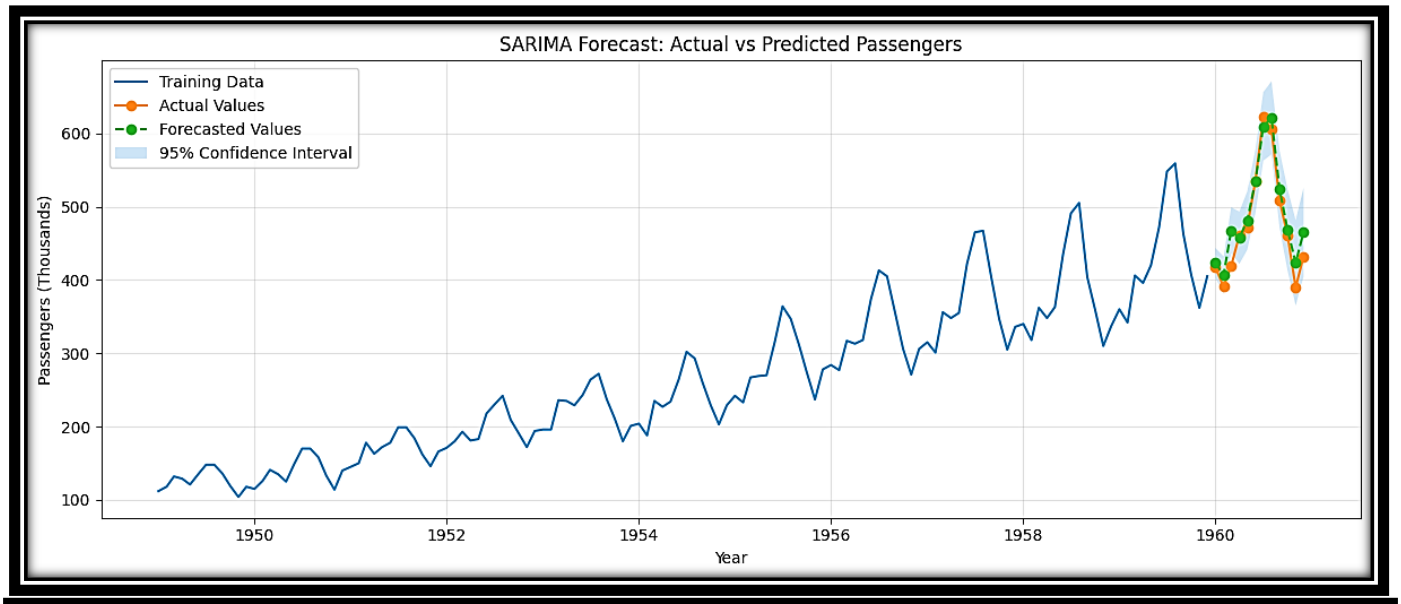

```
Training Samples: 132
Testing Samples: 12
```

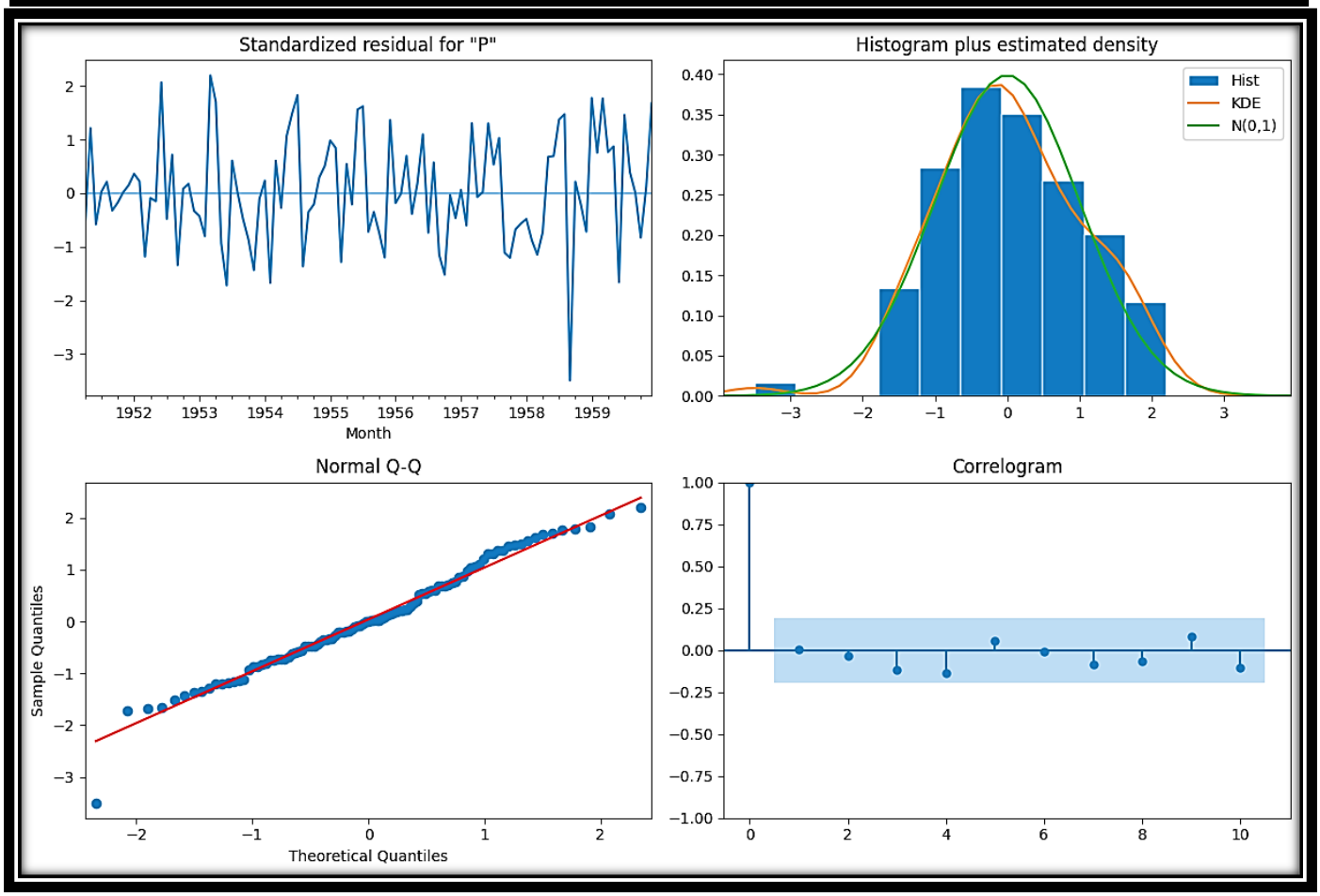
```
SARIMA Model Summary:
                                     SARIMAX Results
==========================================================================================
Dep. Variable:                          Passengers   No. Observations:                  132
Model:             SARIMAX(1, 1, 1)x(1, 1, 1, 12)   Log Likelihood                -396.681
Date:                            Mon, 26 Oct 2026   AIC                            803.363
Time:                                    17:11:29   BIC                            816.633
Sample:                                01-01-1949   HQIC                           808.740
                                     - 12-01-1959
Covariance Type:                              opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5400      0.338     -1.596      0.110      -1.203       0.123
ma.L1          0.3075      0.368      0.835      0.404      -0.414       1.029
ar.S.L12      -0.5925      0.204     -2.907      0.004      -0.992      -0.193
ma.S.L12       0.5235      0.291      1.801      0.072      -0.046       1.093
sigma2       108.4141     17.374      6.240      0.000      74.361     142.467
===================================================================================
Ljung-Box (L1) (Q):                   0.01   Jarque-Bera (JB):                 0.86
Prob(Q):                              0.93   Prob(JB):                         0.65
Heteroskedasticity (H):               1.55   Skew:                            -0.14
Prob(H) (two-sided):                  0.20   Kurtosis:                         3.34
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).

Actual vs Predicted Passenger Counts:
            Actual  Predicted  Lower Bound  Upper Bound
Month
1960-01-01     417     423.22       402.81       443.63
1960-02-01     391     406.43       380.71       432.16
1960-03-01     419     467.55       436.02       499.07
1960-04-01     461     457.48       421.74       493.22
1960-05-01     472     480.94       441.10       520.77
1960-06-01     535     534.60       491.22       577.98
1960-07-01     622     609.41       562.67       656.16
1960-08-01     606     621.01       571.17       670.85
1960-09-01     508     523.36       470.59       576.13
1960-10-01     461     468.64       413.10       524.18
1960-11-01     390     423.34       365.16       481.52
1960-12-01     432     465.07       404.37       525.78

Model Evaluation:
Mean Absolute Error (MAE): 16.67
Root Mean Squared Error (RMSE): 21.62
```





Learning Outcome:

PRACTICAL NO. 9

Aim: Compress and quantize large ML models to make them suitable for deployment on resource constrained devices.

Theory:

Introduction to Model Compression: - Machine Learning and Deep Learning models are widely used in applications such as image recognition, speech processing and computer vision. However, complex models often require considerable memory, storage and computational power. Deploying such models on resource-constrained devices like smartphones, IoT devices and embedded systems can be challenging. Model compression is a technique used to reduce the size and computational requirements of trained models while maintaining their predictive performance as much as possible. It helps make machine learning applications more efficient, portable and suitable for edge-device deployment.

Model Compression Techniques: - Model compression involves reducing the storage and computational requirements of a trained machine learning model. Several techniques are used for this purpose. Quantization reduces the numerical precision of model weights and activations. Pruning removes less important weights or connections from a neural network. Knowledge distillation transfers knowledge from a large teacher model to a smaller student model. Weight sharing allows multiple connections to use common weight values. In this experiment, post-training quantization is used to reduce the precision of a trained CNN model and make it more suitable for lightweight deployment.

Quantization and Numerical Precision: - Quantization is the process of converting high-precision numerical values into lower-precision representations. Deep learning models generally use 32-bit floating-point (FP32) values to store their weights. Quantization converts these values into formats such as INT8, which uses only 8 bits. This reduces model storage requirements and can improve inference efficiency on supported hardware. The quantization process uses scale and zero-point values to map floating-point numbers to integers. Although quantization introduces a small approximation error, it can significantly reduce model size while preserving most of the original model's accuracy.

Post-Training Quantization and Calibration: - Post-Training Quantization (PTQ) is a model compression technique performed after the model has been trained. It does not require complete retraining. During this process, the trained model is converted into a lower-precision format. A representative dataset containing sample inputs is used for calibration to determine suitable numerical ranges for weights and intermediate activations. In this experiment, a subset of Fashion-MNIST training images is used for calibration, and the model is converted into an INT8 quantized format. This approach is computationally efficient and useful when deploying trained models on devices with limited resources.

CNN, TensorFlow Lite and Model Evaluation: - A Convolutional Neural Network (CNN) is a deep learning architecture commonly used for image classification. It consists of convolutional, pooling and dense layers that extract image features and predict their corresponding classes. Keras is used to build and train the CNN, while TensorFlow Lite provides an optimized format for model deployment. The original and quantized models are compared using model size, accuracy and inference time. Model size reduction indicates

storage savings, accuracy measures classification performance, and inference time measures the time required to generate predictions. These measurements help determine the effectiveness of quantization.

Dataset Description:

In this experiment, the built-in Fashion-MNIST dataset from TensorFlow/Keras is used. It contains 70,000 grayscale images of clothing and footwear divided into 10 categories. Each image has a resolution of 28 × 28 pixels with a single colour channel. The dataset consists of 60,000 training images and 10,000 testing images. Its categories include T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag and Ankle boot. The pixel values are normalized between 0 and 1 before training. The training dataset is used to train the CNN, while the testing dataset is used to evaluate the original and quantized models.

Algorithm:

1. Import the required libraries, including TensorFlow, NumPy, Matplotlib and time.
2. Load the Fashion-MNIST dataset containing 60,000 training images and 10,000 testing images.
3. Normalize the pixel values between 0 and 1 and reshape the images for CNN input.
4. Build a CNN model using convolutional, pooling, flatten and dense layers.
5. Compile and train the CNN model using the Adam optimizer and categorical classification loss.
6. Evaluate the trained model and convert it into TensorFlow Lite FP32 format.
7. Prepare a representative dataset using sample training images for calibration.
8. Apply post-training INT8 quantization using the TensorFlow Lite Converter.
9. Save the quantized model in .tflite format and load it using the LiteRT interpreter.
10. Compare the FP32 and INT8 models based on file size, classification accuracy and inference time.
11. Display graphical comparisons of model size, accuracy and inference performance.
12. Demonstrate the compressed model and conclude the effectiveness of quantization.

Source Code:

```
import os
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "2"

import time
import numpy as np
import tensorflow as tf
import matplotlib.pyplot as plt

tf.get_logger().setLevel("ERROR")

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.fashion_mnist.load_data()

x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0

x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

class_names = [
    "T-shirt", "Trouser", "Pullover", "Dress", "Coat",
    "Sandal", "Shirt", "Sneaker", "Bag", "Ankle boot"]

print("FASHION-MNIST DATASET")
print("Training Images:", len(x_train))
print("Testing Images:", len(x_test))
```

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(28, 28, 1)),
    tf.keras.layers.Conv2D(32, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Conv2D(64, 3, activation="relu"),
    tf.keras.layers.MaxPooling2D(),
    tf.keras.layers.Flatten(),
    tf.keras.layers.Dense(128, activation="relu"),
    tf.keras.layers.Dense(10, activation="softmax")])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"])

print("\nTRAINING ORIGINAL CNN MODEL")

model.fit(
    x_train,
    y_train,
    epochs=3,
    batch_size=128,
    validation_split=0.1,
    verbose=1)

original_loss, original_accuracy = model.evaluate(
    x_test, y_test, verbose=0)

print("\nOriginal CNN Accuracy:", round(original_accuracy * 100, 2), "%")

converter = tf.lite.TFLiteConverter.from_keras_model(model)
fp32_model = converter.convert()
with open("model_fp32.tflite", "wb") as f:
    f.write(fp32_model)
```

```
def representative_dataset():
    for image in x_train[:300]:
        yield [image[np.newaxis, ...].astype(np.float32)]
print("\nAPPLYING INT8 QUANTIZATION")

converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
converter.representative_dataset = representative_dataset
converter.target_spec.supported_ops = [
    tf.lite.OpsSet.TFLITE_BUILTINS_INT8]
converter.inference_input_type = tf.uint8
converter.inference_output_type = tf.uint8

int8_model = converter.convert()

with open("model_int8.tflite", "wb") as f:
    f.write(int8_model)

fp32_size = os.path.getsize("model_fp32.tflite") / (1024 * 1024)
int8_size = os.path.getsize("model_int8.tflite") / (1024 * 1024)

reduction = ((fp32_size - int8_size) / fp32_size) * 100
compression_ratio = fp32_size / int8_size

print("\nMODEL COMPRESSION RESULTS")
print("FP32 Model Size:", round(fp32_size, 3), "MB")
print("INT8 Model Size:", round(int8_size, 3), "MB")
print("Size Reduction:", round(reduction, 2), "%")
print("Compression Ratio:", round(compression_ratio, 2), ":1")

fp32_interpreter = tf.lite.Interpreter(model_path="model_fp32.tflite")
int8_interpreter = tf.lite.Interpreter(model_path="model_int8.tflite")

fp32_interpreter.allocate_tensors()
int8_interpreter.allocate_tensors()
```

```python
def evaluate_tflite(interpreter, images, labels):
    input_details = interpreter.get_input_details()[0]
    output_details = interpreter.get_output_details()[0]

    correct = 0

    for image, label in zip(images, labels):
        image = image[np.newaxis, ...]

        if input_details["dtype"] == np.uint8:
            scale, zero_point = input_details["quantization"]
            image = np.round(image / scale + zero_point)
            image = np.clip(image, 0, 255).astype(np.uint8)
        else:
            image = image.astype(np.float32)

        interpreter.set_tensor(input_details["index"], image)
        interpreter.invoke()

        output = interpreter.get_tensor(output_details["index"])
        prediction = np.argmax(output)

        if prediction == label:
            correct += 1

    return correct / len(labels)

print("\nEVALUATING BOTH MODELS")

total = 2000

fp32_accuracy = evaluate_tflite(
    fp32_interpreter, x_test[:total], y_test[:total])

int8_accuracy = evaluate_tflite(
    int8_interpreter, x_test[:total], y_test[:total])
```

```
print("\nACCURACY COMPARISON")
print("FP32 Model Accuracy:", round(fp32_accuracy * 100, 2), "%")
print("INT8 Model Accuracy:", round(int8_accuracy * 100, 2), "%")

def benchmark(interpreter, image):
    input_details = interpreter.get_input_details()[0]
    image = image[np.newaxis, ...]
    if input_details["dtype"] == np.uint8:
        scale, zero_point = input_details["quantization"]
        image = np.round(image / scale + zero_point)
        image = np.clip(image, 0, 255).astype(np.uint8)
    else:
        image = image.astype(np.float32)
    def inference():
        interpreter.set_tensor(input_details["index"], image)
        interpreter.invoke()
    for _ in range(10):
        inference()
    start = time.perf_counter()
    for _ in range(100):
        inference()
    return (time.perf_counter() - start) / 100 * 1000

fp32_time = benchmark(fp32_interpreter, x_test[0])
int8_time = benchmark(int8_interpreter, x_test[0])

print("\nINFERENCE PERFORMANCE")
print("FP32 Inference Time:", round(fp32_time, 3), "ms")
print("INT8 Inference Time:", round(int8_time, 3), "ms")
print("\nQUANTIZATION VERIFICATION")
print("FP32 Input Type:", fp32_interpreter.get_input_details()[0]["dtype"])
print("INT8 Input Type:", int8_interpreter.get_input_details()[0]["dtype"])
print("INT8 Output Type:", int8_interpreter.get_output_details()[0]["dtype"])
plt.figure(figsize=(6, 4))
plt.bar(["FP32", "INT8"], [fp32_size, int8_size])
plt.title("Model Size Comparison")
plt.ylabel("Size (MB)")
plt.show()
```

```
plt.figure(figsize=(6, 4))
plt.bar(
    ["FP32", "INT8"],
    [fp32_accuracy * 100, int8_accuracy * 100])
plt.title("Model Accuracy Comparison")
plt.ylabel("Accuracy (%)")
plt.ylim(0, 100)
plt.show()

plt.figure(figsize=(6, 4))
plt.bar(
    ["FP32", "INT8"],
    [fp32_time, int8_time])
plt.title("Inference Time Comparison")
plt.ylabel("Time (milliseconds)")
plt.show()

plt.figure(figsize=(9, 3))

for i in range(5):
    plt.subplot(1, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")
    plt.title(class_names[y_test[i]], fontsize=9)
    plt.axis("off")

plt.suptitle("Fashion-MNIST Sample Images")
plt.tight_layout()
plt.show()

print("\nDEPLOYMENT DEMONSTRATION")
print("Compressed Model: model_int8.tflite")
print("Format: TensorFlow Lite")
print("Quantization: INT8")
print("Runtime: LiteRT")
print("Deployment Target: Resource-Constrained Devices")
print("Model compression completed successfully.")
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-labels-idx1-ubyte.gz
29515/29515 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/train-images-idx3-ubyte.gz
26421880/26421880 ━━━━━━━━━━━━━━━━━━━━ 3s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-labels-idx1-ubyte.gz
5148/5148 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/t10k-images-idx3-ubyte.gz
4422102/4422102 ━━━━━━━━━━━━━━━━━━━━ 1s 0us/step
FASHION-MNIST DATASET
Training Images: 60000
Testing Images: 10000

TRAINING ORIGINAL CNN MODEL
Epoch 1/3
422/422 ━━━━━━━━━━━━━━━━━━━━ 14s 18ms/step - accuracy: 0.7999 - loss: 0.5602 - val_accuracy: 0.8542 - val_loss: 0.3964
Epoch 2/3
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - accuracy: 0.8700 - loss: 0.3607 - val_accuracy: 0.8690 - val_loss: 0.3557
Epoch 3/3
422/422 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step - accuracy: 0.8858 - loss: 0.3157 - val_accuracy: 0.8802 - val_loss: 0.3251

Original CNN Accuracy: 87.68 %
Saved artifact at '/tmp/tmpjam0x3n2'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28, 1), dtype=tf.float32, name='keras_tensor')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  135704677821968: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822928: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822544: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823312: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823120: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822736: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823504: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677824080: TensorSpec(shape=(), dtype=tf.resource, name=None)
```

```
APPLYING INT8 QUANTIZATION
Saved artifact at '/tmp/tmp6o1u4519'. The following endpoints are available:

* Endpoint 'serve'
  args_0 (POSITIONAL_ONLY): TensorSpec(shape=(None, 28, 28, 1), dtype=tf.float32, name='keras_tensor')
Output Type:
  TensorSpec(shape=(None, 10), dtype=tf.float32, name=None)
Captures:
  135704677821968: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822928: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822544: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823312: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823120: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677822736: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677823504: TensorSpec(shape=(), dtype=tf.resource, name=None)
  135704677824080: TensorSpec(shape=(), dtype=tf.resource, name=None)
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/convert.py:846: UserWarning: Statistics for quantized inputs were expected, but not specified; continuing anyway.
  warnings.warn(

MODEL COMPRESSION RESULTS
FP32 Model Size: 0.863 MB
INT8 Model Size: 0.226 MB
Size Reduction: 73.84 %
Compression Ratio: 3.82 :1

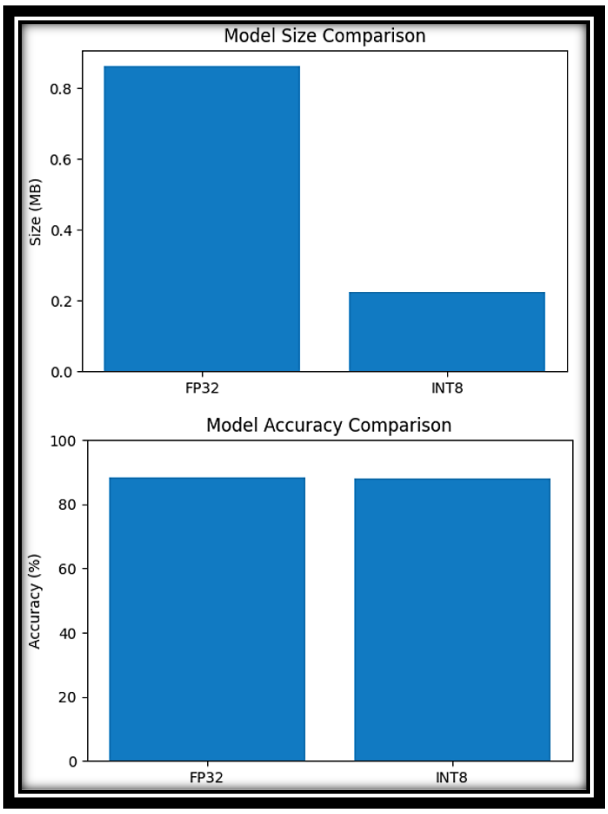
EVALUATING BOTH MODELS
/usr/local/lib/python3.13/dist-packages/tensorflow/lite/python/interpreter.py:457: UserWarning:     Warning: tf.lite.Interpreter is deprecated and is scheduled for deletion in
    TF 2.20. Please use the LiteRT interpreter from the ai_edge_litert package.
    See the [migration guide](https://ai.google.dev/edge/litert/migration)
    for details.

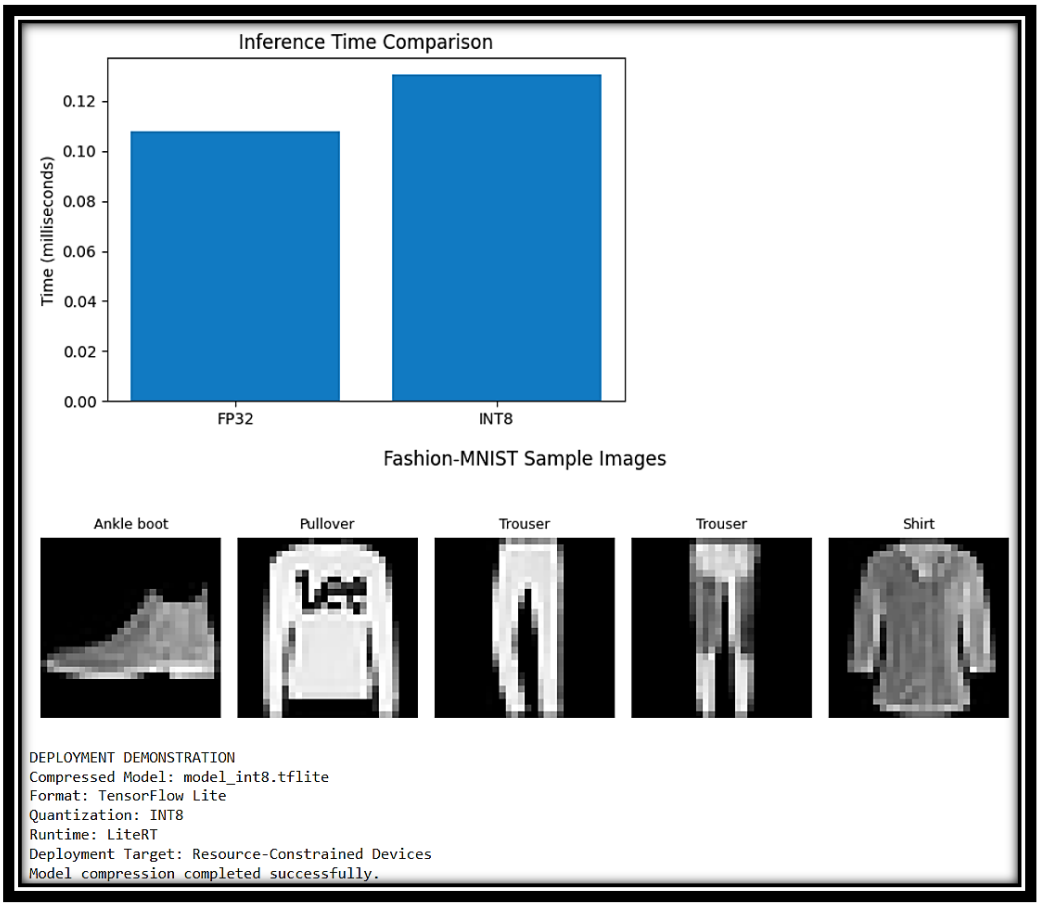
  warnings.warn(_INTERPRETER_DELETION_WARNING)

ACCURACY COMPARISON
FP32 Model Accuracy: 88.3 %
INT8 Model Accuracy: 88.2 %

INFERENCE PERFORMANCE
FP32 Inference Time: 0.108 ms
INT8 Inference Time: 0.131 ms

QUANTIZATION VERIFICATION
FP32 Input Type: <class 'numpy.float32'>
INT8 Input Type: <class 'numpy.uint8'>
INT8 Output Type: <class 'numpy.uint8'>
```





Learning Outcome:

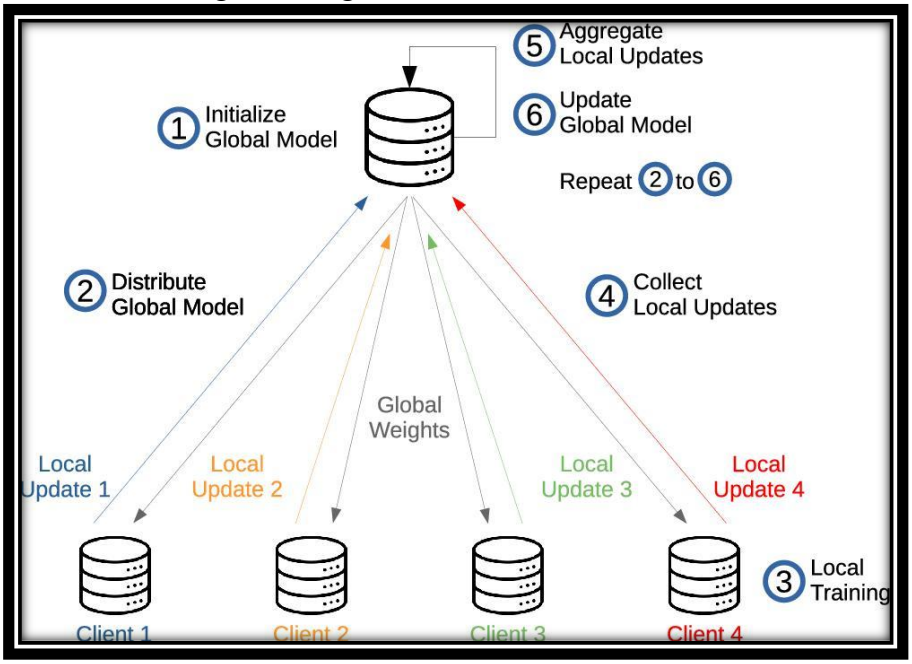
PRACTICAL NO. 10

Aim: Explore federated learning concepts and implement distributed ML models using TensorFlow Federated.

Theory:

Federated Learning is a machine learning technique in which multiple devices or clients collaboratively train a shared machine learning model without directly sharing their original data. Unlike traditional machine learning, where data from different sources is collected and stored on a central server, federated learning allows each client to keep its data locally. The central server distributes a common model to all clients, and each client trains the model using its own dataset. Only the model updates are shared with the server, helping maintain data privacy and reducing the need for transferring large amounts of raw data.

In this experiment, federated learning is implemented using TensorFlow and NumPy to simulate a distributed training environment consisting of five clients. The training dataset is divided equally among the clients, with each client receiving a separate portion of the MNIST dataset. TensorFlow is used to create and train the CNN models, while NumPy is used to aggregate the model weights received from different clients. This approach demonstrates the working of federated learning and allows multiple clients to collaboratively train a common global model without sharing their original data.



The MNIST dataset is used to train a Convolutional Neural Network (CNN) for handwritten digit classification. The dataset contains grayscale images of handwritten digits ranging from 0 to 9, with each image having a resolution of 28 × 28 pixels. In this experiment, 10,000 training images are selected and divided equally among five clients, giving each client 2,000 images. The images are normalized by dividing pixel values by 255 so that they fall within the range of 0 to 1. Each client uses its local dataset to train the model, while the original images and labels remain separate from the other clients.

The experiment uses the Federated Averaging (FedAvg) algorithm to combine learning updates received from different clients. Initially, the server creates a global model and distributes its weights to all participating clients. Each client trains the model locally using its assigned dataset and sends the updated weights back to the server, where they are aggregated to improve the global model. This process is repeated for five communication rounds, using a Convolutional Neural Network (CNN) for handwritten digit classification. The CNN consists of convolutional, max-pooling, flatten, and dense layers, with a final softmax layer to predict the digit classes. Stochastic Gradient Descent (SGD) with momentum is used as the client optimizer. The training accuracy and loss are recorded after each round, and the final global model is evaluated using the MNIST testing dataset. Graphs are generated to observe the model's performance across communication rounds. Federated Learning is useful in applications such as healthcare, banking, mobile applications, and personalized recommendation systems, where data privacy and distributed learning are important. It allows multiple devices or organizations to collaboratively train models without sharing their original datasets, providing an effective approach to privacy-preserving machine learning.

Dataset Description:

The MNIST dataset is used in this experiment to perform handwritten digit classification using federated learning. It contains 70,000 grayscale images of handwritten digits from 0 to 9, divided into 60,000 training images and 10,000 testing images. Each image has a resolution of 28 × 28 pixels. In this experiment, 10,000 training images are selected and distributed equally among five clients, with each client receiving 2,000 images. The pixel values are normalized between 0 and 1 before being used for training. The testing dataset contains 10,000 images and is used to evaluate the final global model after federated training. The dataset is suitable for this experiment because it is widely used for image classification and provides a simple way to demonstrate collaborative learning among multiple clients.

Algorithm:

1. Import the required libraries, including TensorFlow, NumPy, and Matplotlib.
2. Load the MNIST dataset containing handwritten digit images.
3. Select 10,000 training images and normalize their pixel values between 0 and 1.
4. Divide the selected training data equally among five clients, with 2,000 images per client.
5. Create a CNN model consisting of convolutional, max-pooling, flatten, and dense layers.
6. Initialize the global model.
7. Distribute the global model weights among all five clients.
8. Each client creates a local model and loads the global model weights.
9. Train each local model using its own dataset for one epoch with SGD and momentum.
10. Collect the updated model weights from all clients.
11. Aggregate the client model weights using the Federated Averaging algorithm.
12. Update the global model with the averaged weights.
13. Repeat the training and aggregation process for five communication rounds.
14. Evaluate the final global model using the MNIST testing dataset.
15. Display the final test accuracy and loss.
16. Plot the training accuracy and loss across communication rounds.
17. Display sample handwritten digit images with their predicted labels.

Source Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
tf.random.set_seed(42)

(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()

x_train = x_train[:10000].astype("float32") / 255.0
y_train = y_train[:10000]
x_test = x_test.astype("float32") / 255.0

x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

num_clients = 5
client_data = []
indices = np.random.permutation(len(x_train))

for i in range(num_clients):
    idx = indices[i * 2000:(i + 1) * 2000]
    client_data.append((x_train[idx], y_train[idx]))

def create_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28, 1)),
        tf.keras.layers.Conv2D(16, (3, 3), activation="relu"),
        tf.keras.layers.MaxPooling2D((2, 2)),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(64, activation="relu"),
        tf.keras.layers.Dense(10, activation="softmax")])
    model.compile(
        optimizer=tf.keras.optimizers.SGD(learning_rate=0.02, momentum=0.9),
        loss="sparse_categorical_crossentropy",
        metrics=["accuracy"])
    return model
```

```
global_model = create_model()
accuracy_history = []
loss_history = []

for round_num in range(1, 6):
    client_weights = []
    client_sizes = []

    global_weights = global_model.get_weights()

    for x_client, y_client in client_data:
        local_model = create_model()
        local_model.set_weights(global_weights)

        local_model.fit(
            x_client, y_client,
            epochs=1,
            batch_size=32,
            verbose=0)

        client_weights.append(local_model.get_weights())
        client_sizes.append(len(x_client))

    total_samples = sum(client_sizes)

    averaged_weights = [
        np.average(
            [client_weights[i][layer] for i in range(num_clients)],
            axis=0,
            weights=client_sizes)for layer in range(len(global_weights))]

    global_model.set_weights(averaged_weights)
    loss, accuracy = global_model.evaluate(x_test, y_test, verbose=0)
    accuracy_history.append(accuracy)
    loss_history.append(loss)
    print("Round:", round_num,
          "Accuracy:", round(accuracy, 4),
          "Loss:", round(loss, 4))
```

```
test_loss, test_accuracy = global_model.evaluate(x_test, y_test, verbose=0)
print("\nFinal Test Accuracy:", round(test_accuracy, 4))
print("Final Test Loss:", round(test_loss, 4))

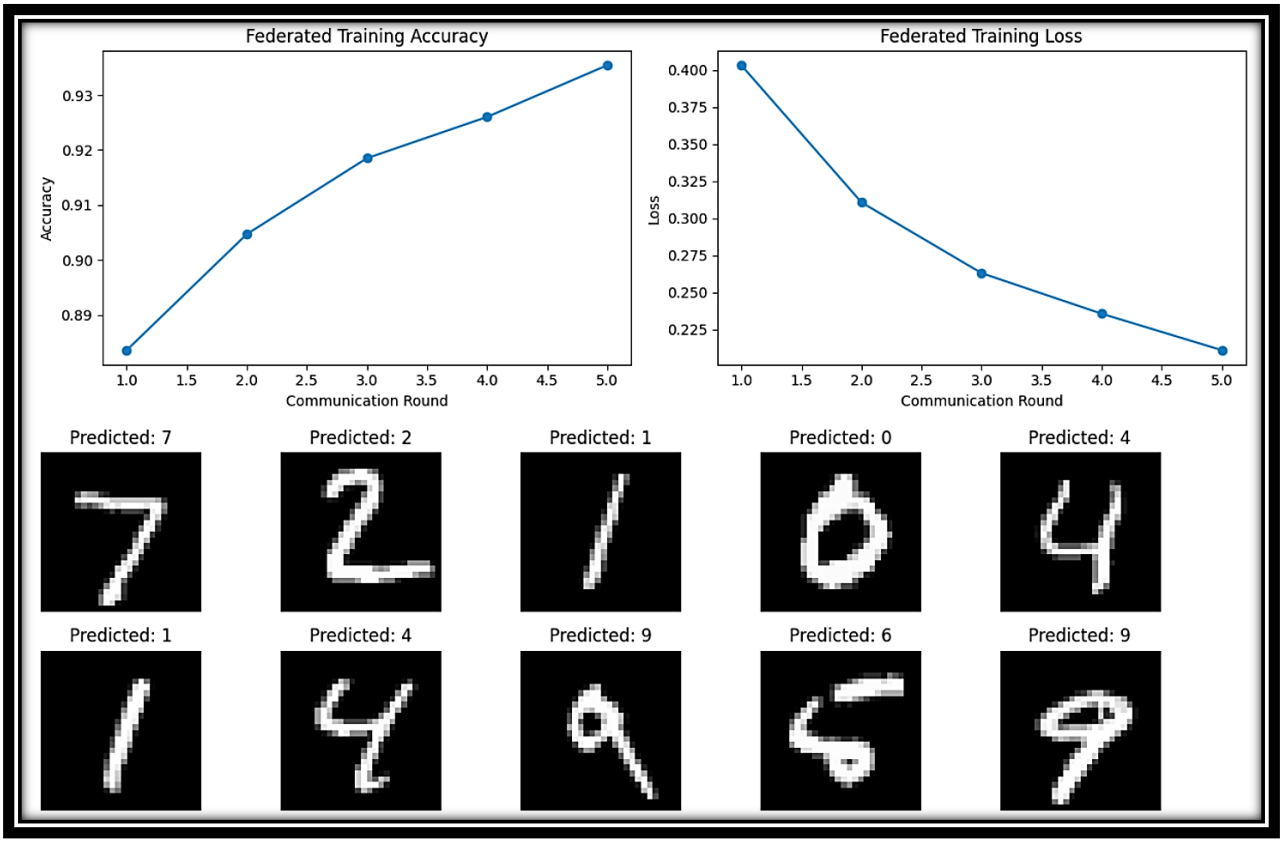
plt.figure(figsize=(12, 4))

plt.subplot(1, 2, 1)
plt.plot(range(1, 6), accuracy_history, marker="o")
plt.title("Federated Training Accuracy")
plt.xlabel("Communication Round")
plt.ylabel("Accuracy")
plt.subplot(1, 2, 2)
plt.plot(range(1, 6), loss_history, marker="o")
plt.title("Federated Training Loss")
plt.xlabel("Communication Round")
plt.ylabel("Loss")
plt.tight_layout()
plt.show()
predictions = global_model.predict(x_test[:10], verbose=0)
plt.figure(figsize=(12, 4))
for i in range(10):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")
    plt.title("Predicted: " + str(np.argmax(predictions[i])))
    plt.axis("off")
plt.tight_layout()
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Round: 1 Accuracy: 0.8836 Loss: 0.4033
Round: 2 Accuracy: 0.9047 Loss: 0.3108
Round: 3 Accuracy: 0.9185 Loss: 0.2631
Round: 4 Accuracy: 0.926 Loss: 0.2357
Round: 5 Accuracy: 0.9354 Loss: 0.2111

Final Test Accuracy: 0.9354
Final Test Loss: 0.2111
```



Learning Outcome:

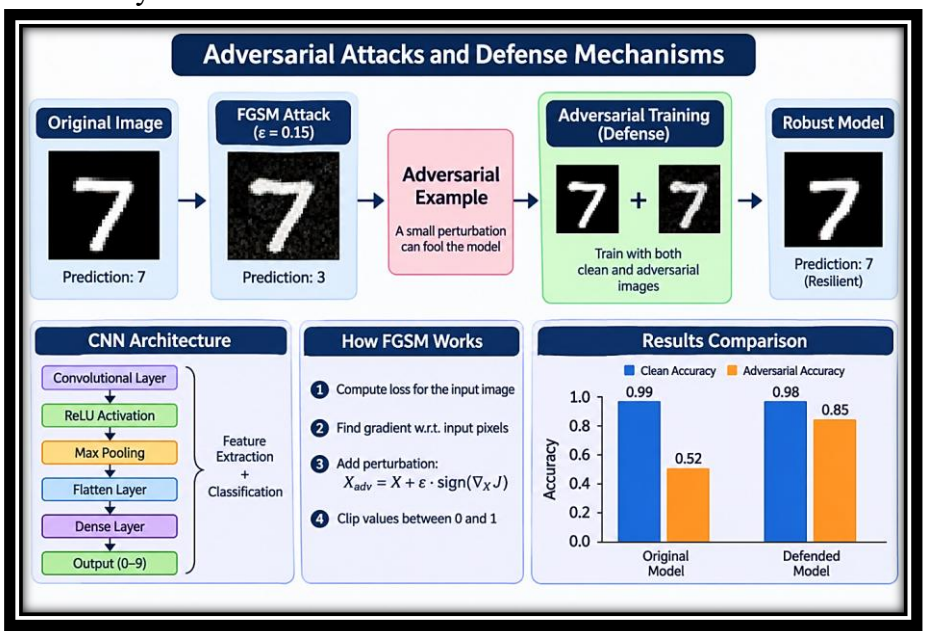
PRACTICAL NO. 11

Aim: Generate adversarial attacks on ML models and implement defense mechanisms to enhance model robustness.

Theory:

Adversarial attacks are techniques used to intentionally modify input data to cause a machine learning model to make incorrect predictions. The modified input is known as an adversarial example. These modifications are usually small and may be difficult for humans to notice, but they can significantly affect the model's output. For example, a handwritten digit image may look almost identical to the original image, but after adding small changes to its pixels, a trained model may classify it as a different digit. Adversarial attacks are important to study because they reveal weaknesses in machine learning models and help evaluate their reliability.

In this experiment, the MNIST dataset is used to perform handwritten digit classification using a Convolutional Neural Network (CNN). A CNN is a deep learning architecture commonly used for image recognition tasks. It consists of convolutional layers that extract important features such as edges and shapes, max-pooling layers that reduce feature dimensions, flatten layers that convert feature maps into one-dimensional data, and dense layers that perform classification. The final output layer contains ten neurons representing the digits from 0 to 9. The model is first trained using normal images so that its original classification accuracy can be measured.



The experiment uses the Fast Gradient Sign Method (FGSM) to generate adversarial examples. FGSM is a gradient-based attack that calculates how the input image affects the model's prediction loss. TensorFlow's GradientTape is used to calculate the gradient of the loss with respect to the input pixels. The sign of the gradient determines the direction in which each pixel should be modified. A small perturbation controlled by epsilon is then added to the original image to create an adversarial example. The modified pixel values are clipped between 0 and 1 to maintain the valid image range. In this experiment, epsilon is set to 0.15 to control the strength of the attack.

To improve model robustness, adversarial training is used as the defense mechanism. In this method, the model is trained using both original images and their adversarial versions. First, FGSM generates modified images from the training data. The original and adversarial images are then combined, while their original labels are retained because the images still represent the same handwritten digits. The model learns from both types of inputs, helping it become more resistant to adversarial modifications. This process is repeated over multiple training epochs to improve its ability to correctly classify both normal and modified images.

Finally, the original and defended models are evaluated using clean and adversarial test images. Clean accuracy measures the model's performance on normal images, while adversarial accuracy measures its performance on modified images. The results are compared to understand whether adversarial training improves the model's robustness. Original and adversarial images are also displayed to observe the effect of the attack, and an accuracy comparison graph is generated. Adversarial robustness is important in applications such as healthcare, autonomous vehicles, face recognition, and cybersecurity, where incorrect predictions can affect the reliability and safety of machine learning systems.

Dataset Description:

The MNIST dataset is used in this experiment for handwritten digit classification. It contains 70,000 grayscale images of handwritten digits ranging from 0 to 9, divided into 60,000 training images and 10,000 testing images. Each image has a resolution of 28 × 28 pixels, and the dataset contains ten different digit classes. The pixel values originally range from 0 to 255 and are converted into floating-point values between 0 and 1 through normalization. An additional channel dimension is added to make the images compatible with CNN layers. The training dataset is used to train the original model and the defended model, while the testing dataset is used to evaluate their performance on clean and adversarial images. MNIST is suitable for this experiment because it is widely used for image classification and provides a simple way to demonstrate adversarial attacks and defence techniques.

Algorithm:

1. Import the required libraries, including TensorFlow, NumPy, and Matplotlib.
2. Load the MNIST dataset and normalize pixel values between 0 and 1.
3. Add a channel dimension to the training and testing images.
4. Define a CNN model for handwritten digit classification.
5. Train the original CNN model using normal MNIST training images.
6. Evaluate the original model on clean test images and record its accuracy.
7. Generate adversarial images using the FGSM attack.
8. Calculate the loss between the model's predictions and the actual labels.
9. Use GradientTape to calculate gradients with respect to the input images.
10. Add perturbations using the sign of the gradients and the selected epsilon value.
11. Clip the modified pixel values between 0 and 1.
12. Evaluate the original model on adversarial test images and record its accuracy.
13. Create a new CNN model for adversarial training.
14. Generate adversarial images from training batches using FGSM.
15. Combine original and adversarial images with their corresponding labels.
16. Train the defended model using the combined dataset.
17. Evaluate the defended model on clean and adversarial test images.
18. Compare the accuracy of the original and defended models.
19. Display original and adversarial images along with an accuracy comparison graph.

Source Code:

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
tf.random.set_seed(42)
(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()
x_train = x_train.astype("float32") / 255.0
x_test = x_test.astype("float32") / 255.0
x_train = x_train[..., np.newaxis]
x_test = x_test[..., np.newaxis]

def build_model():
    model = tf.keras.Sequential([
        tf.keras.layers.Input(shape=(28, 28, 1)),
        tf.keras.layers.Conv2D(32, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Conv2D(64, 3, activation="relu"),
        tf.keras.layers.MaxPooling2D(),
        tf.keras.layers.Flatten(),
        tf.keras.layers.Dense(128, activation="relu"),
        tf.keras.layers.Dense(10)])
    model.compile(
        optimizer="adam",
        loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
        metrics=["accuracy"])
    return model
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)

def fgsm_attack(model, images, labels, epsilon=0.15):
    images = tf.convert_to_tensor(images)

    with tf.GradientTape() as tape:
        tape.watch(images)
        predictions = model(images, training=False)
        loss = loss_fn(labels, predictions)

    gradients = tape.gradient(loss, images)
    adversarial_images = images + epsilon * tf.sign(gradients)
    return tf.clip_by_value(adversarial_images, 0, 1)
```

```
model = build_model()
model.fit(x_train, y_train, epochs=3, batch_size=128, verbose=1)
clean_loss, clean_accuracy = model.evaluate(x_test, y_test, verbose=0)
x_test_adv = fgsm_attack(model, x_test, y_test, epsilon=0.15)
adv_predictions = np.argmax(model.predict(x_test_adv, verbose=0), axis=1)
adv_accuracy = np.mean(adv_predictions == y_test)
defended_model = build_model()
optimizer = tf.keras.optimizers.Adam()

train_dataset = tf.data.Dataset.from_tensor_slices(
    (x_train, y_train)).shuffle(10000).batch(128).prefetch(tf.data.AUTOTUNE)

for epoch in range(2):
    for images, labels in train_dataset:
        adv_images = fgsm_attack(defended_model, images, labels, epsilon=0.15)
        combined_images = tf.concat([images, adv_images], axis=0)
        combined_labels = tf.concat([labels, labels], axis=0)
        with tf.GradientTape() as tape:
            predictions = defended_model(combined_images, training=True)
            loss = loss_fn(combined_labels, predictions)
        gradients = tape.gradient(loss, defended_model.trainable_variables)
        optimizer.apply_gradients(zip(gradients, defended_model.trainable_variables))

    print("Defense Epoch:", epoch + 1, "Loss:", round(float(loss), 4))

_, defended_clean_accuracy = defended_model.evaluate(
    x_test, y_test, verbose=0)

x_test_adv_defended = fgsm_attack(
    defended_model, x_test, y_test, epsilon=0.15)

defended_adv_predictions = np.argmax(
    defended_model.predict(x_test_adv_defended, verbose=0), axis=1)

defended_adv_accuracy = np.mean(defended_adv_predictions == y_test)

print("\nModel Performance")
print("Original Clean Accuracy:", round(clean_accuracy, 4))
print("Original Adversarial Accuracy:", round(adv_accuracy, 4))
print("Defended Clean Accuracy:", round(defended_clean_accuracy, 4))
print("Defended Adversarial Accuracy:", round(defended_adv_accuracy, 4))
print("Robustness Improvement:", round(defended_adv_accuracy - adv_accuracy, 4))
```

```
plt.figure(figsize=(10, 5))

for i in range(5):
    plt.subplot(2, 5, i + 1)
    plt.imshow(x_test[i].squeeze(), cmap="gray")
    plt.title("Original: " + str(y_test[i]))
    plt.axis("off")

    plt.subplot(2, 5, i + 6)
    plt.imshow(x_test_adv[i].numpy().squeeze(), cmap="gray")
    plt.title("Attack: " + str(adv_predictions[i]))
    plt.axis("off")

plt.tight_layout()
plt.show()

labels = ["Clean", "Adversarial", "Defended Clean", "Defended Adversarial"]
accuracies = [
    clean_accuracy,
    adv_accuracy,
    defended_clean_accuracy,
    defended_adv_accuracy]

plt.figure(figsize=(9, 4))
plt.bar(labels, accuracies)
plt.ylabel("Accuracy")
plt.ylim(0, 1)
plt.title("Model Robustness Before and After Defense")
plt.xticks(rotation=15)
plt.tight_layout()
plt.show()
```

Output:

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Epoch 1/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 16s 13ms/step - accuracy: 0.9412 - loss: 0.2025
Epoch 2/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9838 - loss: 0.0530
Epoch 3/3
469/469 ━━━━━━━━━━━━━━━━━━━━ 2s 4ms/step - accuracy: 0.9887 - loss: 0.0372
Defense Epoch: 1 Loss: 0.1347
Defense Epoch: 2 Loss: 0.1983

Model Performance
Original Clean Accuracy: 0.9858
Original Adversarial Accuracy: 0.6293
Defended Clean Accuracy: 0.9851
Defended Adversarial Accuracy: 0.9173
Robustness Improvement: 0.288
```



Learning Outcome:

PRACTICAL NO. 12

Aim: Utilize Ray Tune to perform hyperparameter search and optimize ML models efficiently.

Theory:

Ray Tune: - Ray Tune is a Python library provided by the Ray ecosystem for performing automated hyperparameter optimization. It helps evaluate different hyperparameter configurations, manage multiple training experiments and identify the best-performing configuration. Instead of manually changing hyperparameters and training the model repeatedly, Ray Tune automates this process.

The user defines the hyperparameters and their possible values or ranges. Ray Tune then selects combinations, executes individual training trials and compares their performance. It also supports parallel execution, resource management and early stopping of less promising trials. This makes hyperparameter tuning more organized and efficient, especially when working with machine learning models that have several configurable settings.

Parameters and Hyperparameters: - Parameters are values that a machine learning model learns automatically from training data. For example, the weights and biases of a neural network are learned during training. In contrast, hyperparameters are settings selected before training that control how the learning algorithm works. Examples include the number of trees in a Random Forest, the maximum depth of a Decision Tree, the learning rate and the number of neighbors in KNN.

The main difference is that model parameters are learned from the data, whereas hyperparameters are selected by the user or an optimization process. Choosing suitable hyperparameters is important because they influence model performance, training time and the possibility of overfitting or underfitting.

Hyperparameter Tuning and Search Space: - Hyperparameter tuning is the process of testing different hyperparameter combinations to find a configuration that provides the best model performance. A search space defines the possible values or ranges that can be explored during this process. For example, a Random Forest model can be tested with different numbers of trees, maximum depths and minimum sample requirements. Ray Tune uses the defined search space to create different configurations called trials. Each trial represents one model training and evaluation run using a particular combination of hyperparameters. The performance of each trial is measured using a selected evaluation metric, such as accuracy. The configuration that achieves the highest validation accuracy can then be selected for training the final optimized model.

Search Algorithms and Scheduling: - A search algorithm determines how hyperparameter combinations are selected from the search space. Grid Search evaluates every specified combination, while Random Search selects combinations randomly. Bayesian Optimization uses the results of previous trials to guide the selection of future configurations. Ray Tune supports different search algorithms, allowing users to choose an approach based on their optimization requirements.

A scheduler controls how trials are executed and can stop poorly performing trials early to save computational resources. One example is ASHA (Asynchronous Successive Halving Algorithm), which allocates resources to promising trials and stops weaker ones. Ray Tune also supports parallel execution of trials and resource allocation across CPUs and GPUs. In this experiment, a limited number of configurations are evaluated to demonstrate hyperparameter optimization efficiently.

Random Forest and Optimization: - Random Forest is an ensemble learning algorithm that combines multiple Decision Trees to make predictions. Each tree is trained using randomly selected samples and features. For classification, the predictions from individual trees are combined through majority voting to produce the final class.

The performance of a Random Forest depends on hyperparameters such as n_estimators, max_depth, min_samples_split and min_samples_leaf. These control the number of trees, the maximum depth of each tree and the minimum number of samples required for splitting and forming leaf nodes. Ray Tune tests different combinations of these settings and identifies the configuration with the highest validation accuracy. The optimized model is then evaluated on the test dataset and compared with the baseline model to observe the effect of hyperparameter tuning.

Dataset Description:

The Wisconsin Breast Cancer dataset is a commonly used dataset for binary classification. It contains 569 samples and 30 numerical features extracted from digitized images of breast tumor cell nuclei. These features describe characteristics such as radius, texture, perimeter, area, smoothness, compactness and symmetry. The objective is to predict whether a tumor is malignant or benign. The dataset contains 212 malignant samples and 357 benign samples. The target classes are encoded as 0 for malignant and 1 for benign. This dataset is suitable for demonstrating hyperparameter optimization because it provides numerical features, two clearly defined classes and a manageable number of samples. In this experiment, the data is divided into training, validation and testing sets. The validation set is used by Ray Tune to compare different hyperparameter configurations, while the test set is used for the final performance evaluation.

Algorithm:

1. Load the Wisconsin Breast Cancer dataset.
2. Divide the dataset into training, validation and testing sets.
3. Initialize a Random Forest Classifier with default hyperparameters.
4. Train the baseline model and calculate its test accuracy.
5. Define a search space for the Random Forest hyperparameters.
6. Create a training function that trains the model and calculates validation accuracy.
7. Initialize Ray and configure Ray Tune with the search space and evaluation metric.
8. Execute multiple hyperparameter search trials.
9. Compare the validation accuracy obtained from different configurations.
10. Select the configuration with the highest validation accuracy.
11. Train a final Random Forest model using the best hyperparameters.
12. Evaluate the optimized model on the test dataset.
13. Compare baseline and optimized model accuracy using a bar graph.
14. Display the best hyperparameters and performance results.

Source Code:

```
!pip -q install "ray[tune]"

import ray
from ray import tune
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_breast_cancer()
X = data.data
y = data.target

X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

X_train, X_val, y_train, y_val = train_test_split(
    X_train_full, y_train_full,
    test_size=0.2, random_state=42, stratify=y_train_full)

baseline_model = RandomForestClassifier(random_state=42, n_jobs=1)
baseline_model.fit(X_train_full, y_train_full)

baseline_pred = baseline_model.predict(X_test)
baseline_accuracy = accuracy_score(y_test, baseline_pred)

def train_model(config, X_train, y_train, X_val, y_val):
    model = RandomForestClassifier(
        n_estimators=config["n_estimators"],
        max_depth=config["max_depth"],
        min_samples_split=config["min_samples_split"],
        min_samples_leaf=config["min_samples_leaf"],
        random_state=42,
        n_jobs=1)
```

```
    model.fit(X_train, y_train)
    predictions = model.predict(X_val)

    return {"val_accuracy": accuracy_score(y_val, predictions)}

search_space = {
    "n_estimators": tune.choice([50, 100, 150, 200]),
    "max_depth": tune.choice([5, 10, 15, 20, None]),
    "min_samples_split": tune.choice([2, 5, 10]),
    "min_samples_leaf": tune.choice([1, 2, 4])}

ray.init(
    num_cpus=2,
    include_dashboard=False,
    ignore_reinit_error=True,
    logging_level="ERROR")

try:
    tuner = tune.Tuner(
        tune.with_parameters(
            train_model,
            X_train=X_train,
            y_train=y_train,
            X_val=X_val,
            y_val=y_val),
        param_space=search_space,
        tune_config=tune.TuneConfig(
            metric="val_accuracy",
            mode="max",
            num_samples=15,
            max_concurrent_trials=2),
        run_config=ray.tune.RunConfig(verbose=0))

    results = tuner.fit()

    best_result = results.get_best_result(
        metric="val_accuracy",
        mode="max")
```

```
    best_config = best_result.config
    best_val_accuracy = best_result.metrics["val_accuracy"]

    best_model = RandomForestClassifier(
        **best_config,
        random_state=42,
        n_jobs=1)

    best_model.fit(X_train_full, y_train_full)
    best_pred = best_model.predict(X_test)
    best_accuracy = accuracy_score(y_test, best_pred)
    print("Best Hyperparameters:")
    print(best_config)
    print("\nModel Performance")
    print(f"Baseline Accuracy: {baseline_accuracy:.4f}")
    print(f"Best Validation Accuracy: {best_val_accuracy:.4f}")
    print(f"Optimized Test Accuracy: {best_accuracy:.4f}")
    plt.figure(figsize=(6, 4))
    plt.bar(
        ["Baseline", "Optimized"],
        [baseline_accuracy * 100, best_accuracy * 100])
    plt.ylabel("Accuracy (%)")
    plt.title("Baseline vs Optimized Random Forest")
    plt.ylim(0, 100)
    plt.tight_layout()
    plt.show()
finally:
    ray.shutdown()
```

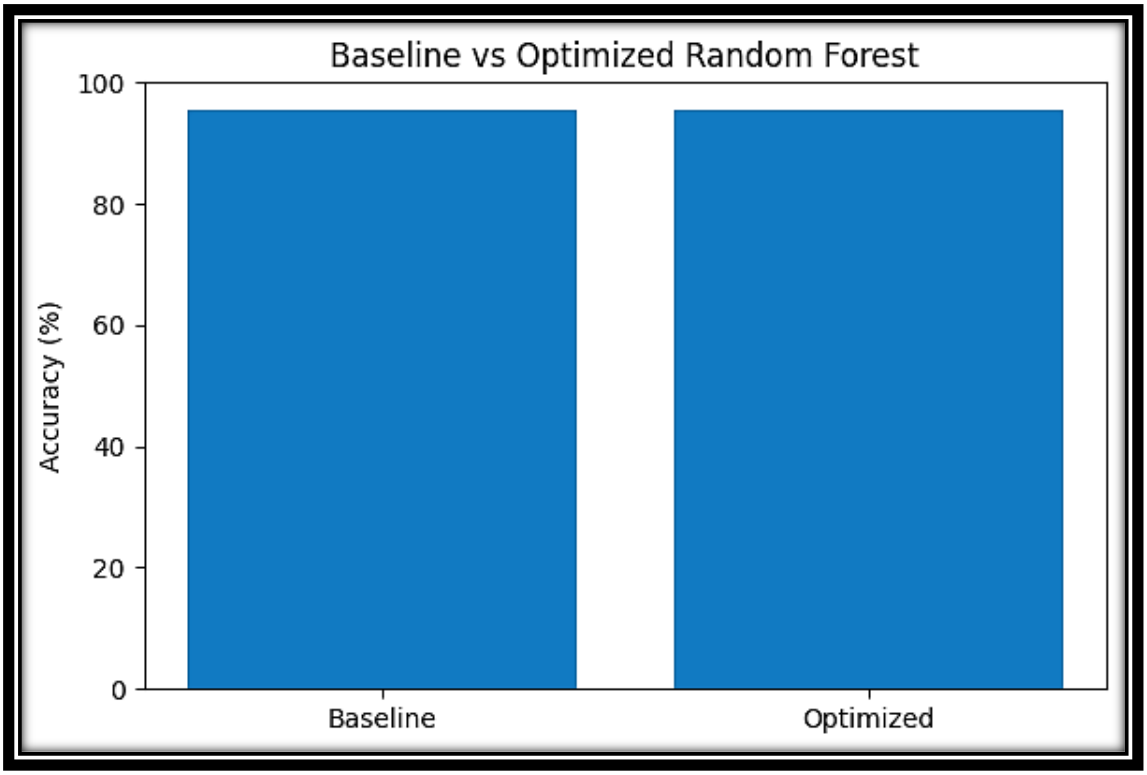
Output:

```
+-----------------------------------------------------------------+
| Configuration for experiment     train_model_2026-11-02_18-47-39   |
+-----------------------------------------------------------------+
| Search algorithm                 BasicVariantGenerator            |
| Scheduler                        FIFOScheduler                    |
| Number of trials                 15                               |
+-----------------------------------------------------------------+

View detailed results here: /root/ray_results/train_model_2026-11-02_18-47-39
To visualize your results with TensorBoard, run: `tensorboard --logdir /tmp/ray/session_2026-11-02_18-47-23_410349_1501/artifacts/2026-11-02_18-47-39/train_model_2026-11-02_18-

(train_model pid=2745) Exception ignored in: <function WeakSet.__init__.<locals>._remove at 0x7e40b601d9e0>
(train_model pid=2745) Traceback (most recent call last):
(train_model pid=2745)   File "/usr/lib/python3.13/_weakrefset.py", line 39, in _remove
(train_model pid=2745)     def _remove(item, selfref=ref(self)):
(train_model pid=2745)   File "/usr/local/lib/python3.13/dist-packages/ray/_private/worker.py", line 1063, in sigterm_handler
(train_model pid=2745)     raise_sys_exit_with_custom_error_message(
(train_model pid=2745)   File "python/ray/_raylet.pyx", line 714, in ray._raylet.raise_sys_exit_with_custom_error_message
(train_model pid=2745) SystemExit
(train_model pid=2745) : 1
Best Hyperparameters:
{'n_estimators': 50, 'max_depth': 10, 'min_samples_split': 2, 'min_samples_leaf': 4}

Model Performance
Baseline Accuracy: 0.9561
Best Validation Accuracy: 0.9560
Optimized Test Accuracy: 0.9561
```



Learning Outcome: